

Now we'll cause the UI to adapt dynamically to a changing window size, and so that it looks good on devices with small displays. To do this, you'll add a [VisualStateManager](#) section into `MainPage.xaml`. You'll define different visual states for different window sizes, and then set the properties to apply for each of those visual states.

Adjust the UI layout

Add this block of XAML as the first child element of the root **StackPanel** element.

XAML

```
<StackPanel ...>
    <VisualStateManager.VisualStateGroups>
        <VisualStateGroup>
            <VisualState x:Name="wideState">
                <VisualState.StateTriggers>
                    <AdaptiveTrigger MinWindowWidth="641" />
                </VisualState.StateTriggers>
            </VisualState>
            <VisualState x:Name="narrowState">
                <VisualState.StateTriggers>
                    <AdaptiveTrigger MinWindowWidth="0" />
                </VisualState.StateTriggers>
                <VisualState.Setters>
                    <Setter Target="contentPanel.Margin" Value="20,30,0,0"/>
                    <Setter Target="inputPanel.Orientation"
Value="Vertical"/>
                    <Setter Target="inputButton.Margin" Value="0,4,0,0"/>
                </VisualState.Setters>
            </VisualState>
        </VisualStateGroup>
    </VisualStateManager.VisualStateGroups>
    ...
</StackPanel>
```

Build and run the app. Notice that the UI looks the same as before until the window is resized narrower than 641 device-independent pixels (DIPs). At that point, the *narrowState* visual state is applied and, along with it, all of the property setters defined for that state.

The [VisualState](#) named `wideState` has an [AdaptiveTrigger](#) with its [MinWindowWidth](#) property set to 641. This means that the state is to be applied only when the window width is not less than the minimum of 641 DIPs. You don't define any [Setter](#) objects for this state, so it uses the layout properties that you defined in the XAML for the page content.

The second **VisualState**, `narrowState`, has an **AdaptiveTrigger** with its **MinWindowWidth** property set to 0. This state is applied when the window width is greater than 0, but less than 641 DIPs. At exactly 641 DIPs, `wideState` is in effect. In `narrowState`, you define **Setter** objects to change the layout properties of controls in the UI.

- You reduce the left margin of the *contentPanel* element from 120 to 20.
- You change the **Orientation** of the *inputPanel* element from **Horizontal** to **Vertical**.
- You add a top margin of 4 DIPs to the *inputButton* element.

Summary

This walkthrough showed you how to add content to a Windows Universal app, how to add interactivity, and how to change the UI's appearance.

Create a "Hello, World!" UWP app in C++/CX

Article • 10/26/2022

Important

This tutorial uses C++/CX. Microsoft has released C++/WinRT: an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs. For more information on this language, please see [C++/WinRT](#).

With Microsoft Visual Studio, you can use C++/CX to develop an app that runs on Windows with a UI that's defined in Extensible Application Markup Language (XAML).

Note

This tutorial uses Visual Studio Community 2019. If you are using a different version of Visual Studio, it may look a little different for you.

Before you start

- To complete this tutorial, you must use Visual Studio Community, or one of the non-Community versions of Visual Studio, on a computer that's running Windows. To download, see [Get the tools](#).
- To follow along, you should have a basic understanding of C++/CX, XAML, and the concepts in the [XAML overview](#).
- To follow along, you should use the default window layout in Visual Studio. To reset to the default layout, on the menu bar, choose **Window** > **Reset Window Layout**.

Comparing C++ desktop apps to UWP apps

If you're coming from a background in Windows desktop programming in C++, then you'll probably find that some aspects of writing apps for the UWP are familiar, but other aspects require some learning.

What's the same?

- You can use the STL, the CRT (with some exceptions), and any other C++ library as long as the code calls only Windows functions that are accessible from the Windows Runtime environment.
- If you're accustomed to visual design surfaces, then you can still use the designer built into Microsoft Visual Studio, or you can use the more full-featured Blend for Visual Studio. If you're accustomed to coding UI by hand, then you can hand-code your XAML.
- You're still creating apps that use Windows operating system types and your own custom types.
- You're still using the Visual Studio debugger, profiler, and other development tools.
- You're still creating apps that are compiled to native machine code by the Visual C++ compiler. UWP apps in C++/CX don't execute in a managed runtime environment.

What's new?

- The design principles for UWP apps are very different from those for desktop apps. Window borders, labels, dialog boxes, and so on, are de-emphasized. Content is foremost. Great UWP apps incorporate these principles from the very beginning of the planning stage.
- You're using XAML to define the entire UI. The separation between UI and core program logic is much clearer in a UWP app than in an MFC or Win32 app. Other people can work on the appearance of the UI in the XAML file while you're working on the behavior in the code file.
- You're primarily programming against a new, easy-to-navigate, object-oriented API, the Windows Runtime, although on Windows devices Win32 is still available for some functionality.
- You use C++/CX to consume and create Windows Runtime objects. C++/CX enables C++ exception handling, delegates, events, and automatic reference counting of dynamically created objects. When you use C++/CX, the details of the underlying COM and Windows architecture are hidden from your app code. For more information, see [C++/CX Language Reference](#).
- Your app is compiled into a package that also contains metadata about the types that your app contains, the resources that it uses, and the capabilities that it

requires (file access, internet access, camera access, and so forth).

- In the Microsoft Store and Windows Phone Store your app is verified as safe by a certification process and made discoverable to millions of potential customers.

Hello World Store app in C++/CX

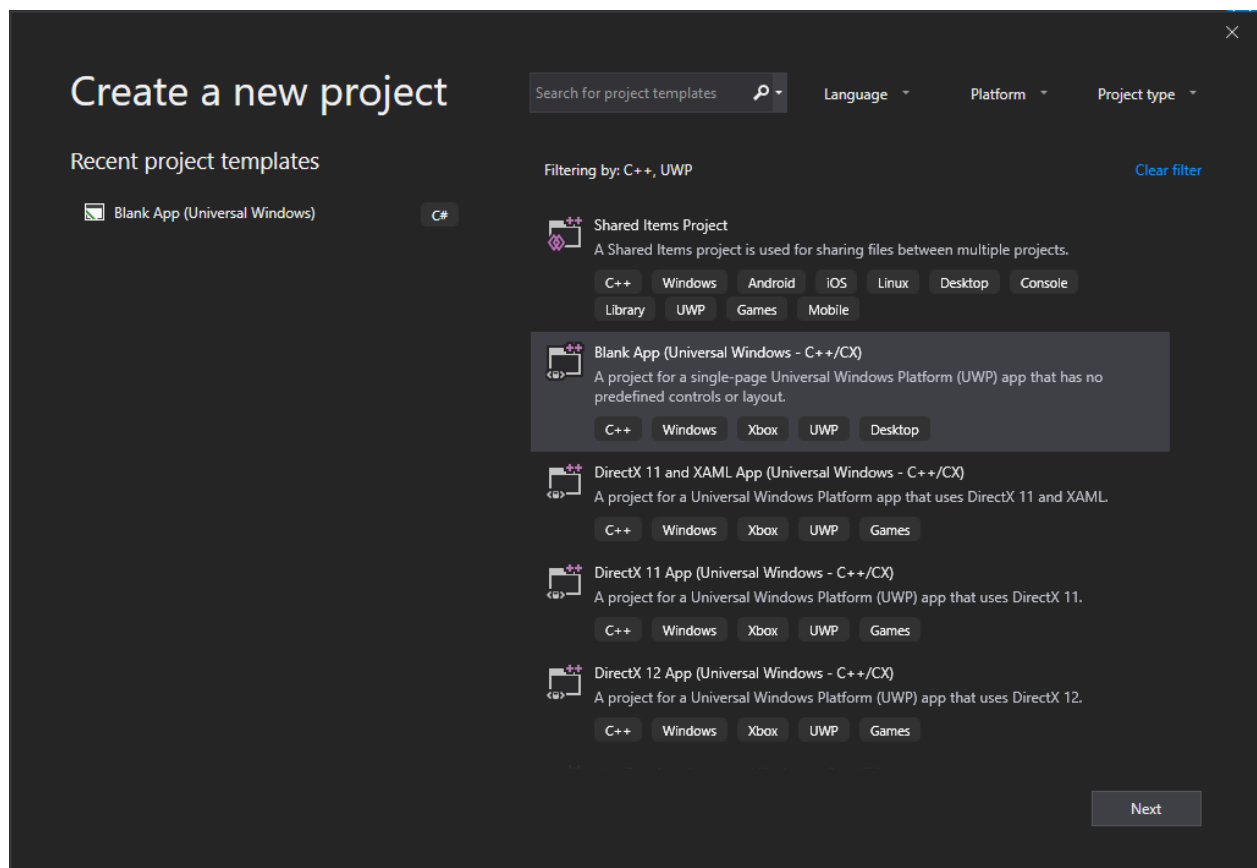
Our first app is a "Hello World" that demonstrates some basic features of interactivity, layout, and styles. We'll create an app from the Windows Universal app project template. If you've developed apps for Windows 8.1 and Windows Phone 8.1 before, you might remember that you had to have three projects in Visual Studio, one for the Windows app, one for the phone app, and another with shared code. The Windows Universal Windows Platform (UWP) makes it possible to have just one project, which runs on all devices, including desktop and laptop computers running Windows, devices such as tablets, mobile phones, VR devices and so on.

We'll start with the basics:

- How to create a Universal Windows project in Visual Studio.
- How to understand the projects and files that are created.
- How to understand the extensions in Visual C++ component extensions (C++/CX), and when to use them.

First, create a solution in Visual Studio

1. In Visual Studio, on the menu bar, choose **File > New > Project....**
2. In the **Create a new project** dialog box, select **Blank App (Universal Windows - C++/CX)**. If you don't see this option, make sure you have the Universal Windows App Development Tools installed. See [Get set up](#) for more information.



3. Choose **Next**, and then enter a name for the project. We'll name it HelloWorld.

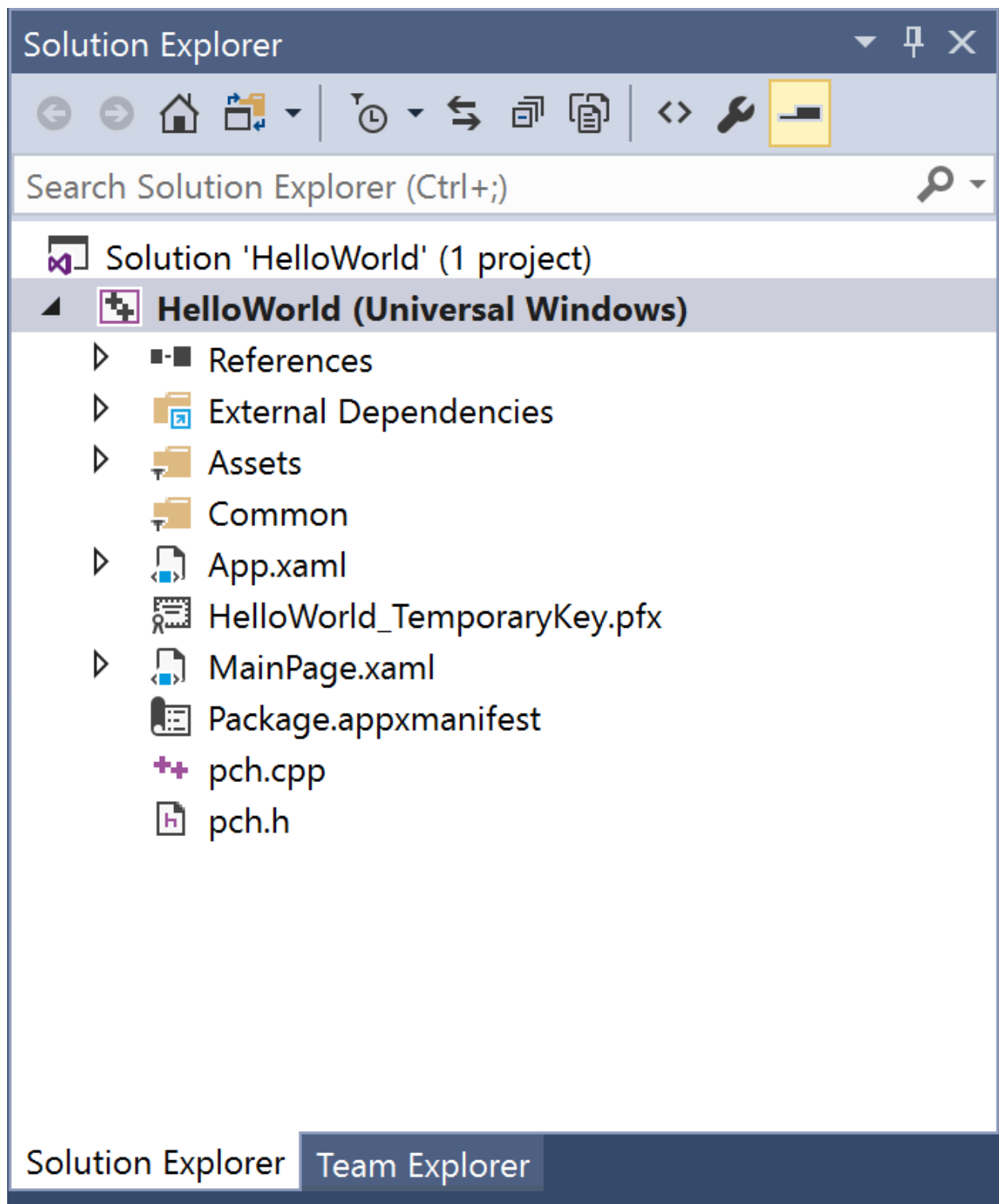
4. Choose the **Create** button.

! Note

If this is the first time you have used Visual Studio, you might see a Settings dialog asking you to enable **Developer mode**. Developer mode is a special setting that enables certain features, such as permission to run apps directly, rather than only from the Store. For more information, please read [Enable your device for development](#). To continue with this guide, select **Developer mode**, click **Yes**, and close the dialog.

Your project files are created.

Before we go on, let's look at what's in the solution.



About the project files

Every .xaml file in a project folder has a corresponding .xaml.h file and .xaml.cpp file in the same folder and a .g file and a .g.hpp file in the Generated Files folder, which is on disk but not part of the project. You modify the XAML files to create UI elements and connect them to data sources (DataBinding). You modify the .h and .cpp files to add custom logic for event handlers. The auto-generated files represent the transformation of the XAML markup into C++/CX. Don't modify these files, but you can study them to better understand how the code-behind works. Basically, the generated file contains a partial class definition for a XAML root element; this class is the same class that you

modify in the *.xaml.h and .cpp files. The generated files declare the XAML UI child elements as class members so that you can reference them in the code you write. At build time, the generated code and your code are merged into a complete class definition and then compiled.

Let's look first at the project files.

- **App.xaml, App.xaml.h, App.xaml.cpp:** Represent the Application object, which is an app's entry point. App.xaml contains no page-specific UI markup, but you can add UI styles and other elements that you want to be accessible from any page. The code-behind files contain handlers for the **OnLaunched** and **OnSuspending** events. Typically, you add custom code here to initialize your app when it starts and perform cleanup when it's suspended or terminated.
- **MainPage.xaml, MainPage.xaml.h, MainPage.xaml.cpp:** Contain the XAML markup and code-behind for the default "start" page in an app. It has no navigation support or built-in controls.
- **pch.h, pch.cpp:** A precompiled header file and the file that includes it in your project. In pch.h, you can include any headers that do not change often and are included in other files in the solution.
- **Package.appxmanifest:** An XML file that describes the device capabilities that your app requires, and the app version info and other metadata. To open this file in the **Manifest Designer**, just double-click it.
- **HelloWorld_TemporaryKey.pfx:** A key that enables deployment of the app on this machine, from Visual Studio.

A first look at the code

If you examine the code in App.xaml.h, App.xaml.cpp in the shared project, you'll notice that it's mostly C++ code that looks familiar. However, some syntax elements might not be as familiar if you are new to Windows Runtime apps, or you've worked with C++/CLI. Here are the most common non-standard syntax elements you'll see in C++/CX:

Ref classes

Almost all Windows Runtime classes, which includes all the types in the Windows API--XAML controls, the pages in your app, the App class itself, all device and network objects, all container types--are declared as a **ref class**. (A few Windows types are **value class** or **value struct**). A ref class is consumable from any language. In C++/CX, the lifetime of these types is governed by automatic reference counting (not garbage collection) so that you never explicitly delete these objects. You can create your own ref classes as well.

C++

```
namespace HelloWorld
{
    /// <summary>
    /// An empty page that can be used on its own or navigated to within a
    Frame.
    /// </summary>
    public ref class MainPage sealed
    {
    public:
        MainPage();
    };
}
```

All Windows Runtime types must be declared within a namespace and unlike in ISO C++ the types themselves have an accessibility modifier. The **public** modifier makes the class visible to Windows Runtime components outside the namespace. The **sealed** keyword means the class cannot serve as a base class. Almost all ref classes are sealed; class inheritance is not broadly used because JavaScript does not understand it.

ref new and ^ (hats)

You declare a variable of a ref class by using the ^ (hat) operator, and you instantiate the object with the ref new keyword. Thereafter you access the object's instance methods with the -> operator just like a C++ pointer. Static methods are accessed with the :: operator just as in ISO C++.

In the following code, we use the fully qualified name to instantiate an object, and use the -> operator to call an instance method.

C++

```
Windows::UI::Xaml::Media::Imaging::BitmapImage^ bitmapImage =
    ref new Windows::UI::Xaml::Media::Imaging::BitmapImage();

bitmapImage->SetSource(fileStream);
```

Typically, in a .cpp file we would add a `using namespace Windows::UI::Xaml::Media::Imaging` directive and the auto keyword, so that the same code would look like this:

C++

```
auto bitmapImage = ref new BitmapImage();
bitmapImage->SetSource(fileStream);
```

Properties

A ref class can have properties, which, just as in managed languages, are special member functions that appear as fields to consuming code.

C++

```
public ref class SaveStateEventArgs sealed
{
    public:
        // Declare the property
        property Windows::Foundation::Collections::IMap<Platform::String^,
Platform::Object^>^ PageState
        {
            Windows::Foundation::Collections::IMap<Platform::String^,
Platform::Object^>^ get();
        }
        ...
};

...
// consume the property like a public field
void PhotoPage::SaveState(Object^ sender, Common::SaveStateEventArgs^ e)
{
    if (mruToken != nullptr && !mruToken->IsEmpty())
    {
        e->PageState->Insert("mruToken", mruToken);
    }
}
```

Delegates

Just as in managed languages, a delegate is a reference type that encapsulates a function with a specific signature. They are most often used with events and event handlers

C++

```
// Delegate declaration (within namespace scope)
public delegate void LoadStateEventHandler(Platform::Object^ sender,
LoadStateEventArgs^ e);

// Event declaration (class scope)
public ref class NavigationHelper sealed
{
    public:
        event LoadStateEventHandler^ LoadState;
};

// Create the event handler in consuming class
MainPage::MainPage()
```

```
{
    auto navigationHelper = ref new Common::NavigationHelper(this);
    navigationHelper->LoadState += ref new
Common::LoadStateEventHandler(this, &MainPage::LoadState);
}
```

Adding content to the app

Let's add some content to the app.

Step 1: Modify your start page

1. In **Solution Explorer**, open MainPage.xaml.
2. Create controls for the UI by adding the following XAML to the root **Grid**, immediately before its closing tag. It contains a **StackPanel** that has a **TextBlock** that asks the user's name, a **TextBox** element that accepts the user's name, a **Button**, and another **TextBlock** element.

XAML

```
<StackPanel x:Name="contentPanel" Margin="120,30,0,0">
    <TextBlock HorizontalAlignment="Left" Text="Hello World"
FontSize="36"/>
    <TextBlock Text="What's your name?"/>
    <StackPanel x:Name="inputPanel" Orientation="Horizontal"
Margin="0,20,0,20">
        <TextBox x:Name="nameInput" Width="300"
HorizontalAlignment="Left"/>
        <Button x:Name="inputButton" Content="Say "Hello""/>
    </StackPanel>
    <TextBlock x:Name="greetingOutput"/>
</StackPanel>
```

3. At this point, you have created a very basic Universal Windows app. To see what the UWP app looks like, press F5 to build, deploy, and run the app in debugging mode.

The default splash screen appears first. It has an image—Assets\SplashScreen.scale-100.png—and a background color that are specified in the app's manifest file. To learn how to customize the splash screen, see [Adding a splash screen](#).

When the splash screen disappears, your app appears. It displays the main page of the App.

Hello, World!

What's your name?

Say "Hello"

It doesn't do much—yet—but congratulations, you've built your first Universal Windows Platform app!

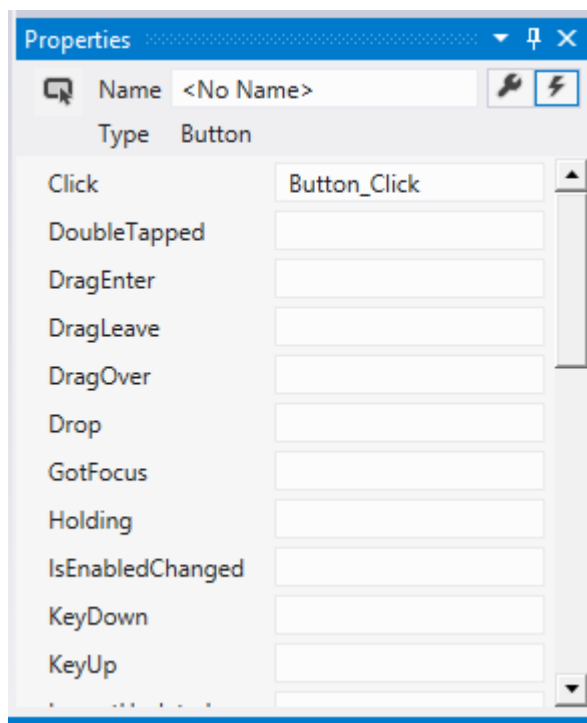
To stop debugging and close the app, return to Visual Studio and press Shift+F5.

For more information, see [Run a Store app from Visual Studio](#).

In the app, you can type in the [TextBox](#), but clicking the [Button](#) doesn't do anything. In later steps, you create an event handler for the button's [Click](#) event, which displays a personalized greeting.

Step 2: Create an event handler

1. In MainPage.xaml, in either XAML or design view, select the "Say Hello" [Button](#) in the [StackPanel](#) you added earlier.
2. Open the **Properties Window** by pressing F4, and then choose the Events button ().
3. Find the [Click](#) event. In its text box, type the name of the function that handles the **Click** event. For this example, type "Button_Click".



4. Press Enter. The event handler method is created in MainPage.xaml.cpp and opened so that you can add the code that's executed when the event occurs.

At the same time, in MainPage.xaml, the XAML for the **Button** is updated to declare the **Click** event handler, like this:

XAML

```
<Button Content="Say "Hello"," Click="Button_Click"/>
```

You could also have simply added this to the xaml code manually, which can be helpful if the designer doesn't load. If you enter this manually, type "Click" and then let IntelliSense pop up the option to add a new event handler. That way, Visual Studio creates the necessary method declaration and stub.

The designer fails to load if an unhandled exception occurs during rendering. Rendering in the designer involves running a design-time version of the page. It can be helpful to disable running user code. You can do this by changing the setting in the **Tools, Options** dialog box. Under **XAML Designer**, uncheck **Run project code in XAML designer** (if supported).

5. In MainPage.xaml.cpp, add the following code to the **Button_Click** event handler that you just created. This code retrieves the user's name from the `nameInput` **TextBox** control and uses it to create a greeting. The `greetingOutput` **TextBlock** displays the result.

C++

```
void HelloWorld::MainPage::Button_Click(Platform::Object^ sender,
Windows::UI::Xaml::RoutedEventArgs^ e)
{
    greetingOutput->Text = "Hello, " + nameInput->Text + "!";
}
```

6. Set the project as the startup, and then press F5 to build and run the app. When you type a name in the text box and click the button, the app displays a personalized greeting.

Hello, world!

What's your name?

Say "Hello"

Step 3: Style the start page

Choosing a theme

It's easy to customize the look and feel of your app. By default, your app uses resources that have a light style. The system resources also include a light theme. Let's try it out and see what it looks like.

To switch to the dark theme

1. Open App.xaml.
2. In the opening **Application** tag, edit the **RequestedTheme** property and set its value to **Dark**:

XAML

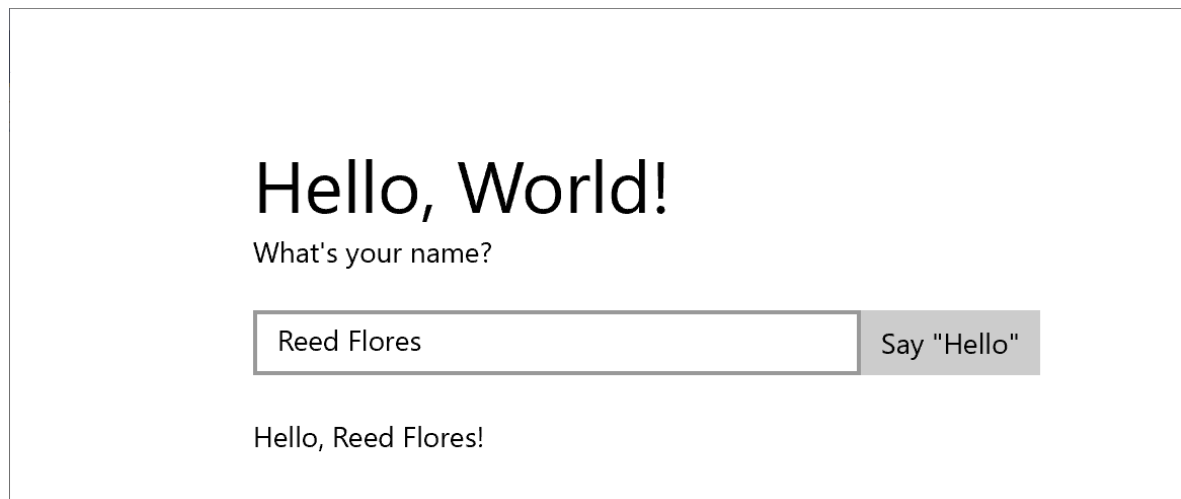
```
RequestedTheme="Dark"
```

Here's the full **Application** tag with the dark theme :

XAML

```
<Application
  x:Class="HelloWorld.App"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:local="using:HelloWorld"
  RequestedTheme="Dark">
```

3. Press F5 to build and run it. Notice that it uses the dark theme.



Hello, World!

What's your name?

Reed Flores Say "Hello"

Hello, Reed Flores!

Which theme should you use? Whichever one you want. Here's our take: for apps that mostly display images or video, we recommend the dark theme; for apps that contain a lot of text, we recommend the light theme. If you're using a custom color scheme, use the theme that goes best with your app's look and feel. In the rest of this tutorial, we use the Light theme in screenshots.

Note The theme is applied when the app is started and can't be changed while the app is running.

Using system styles

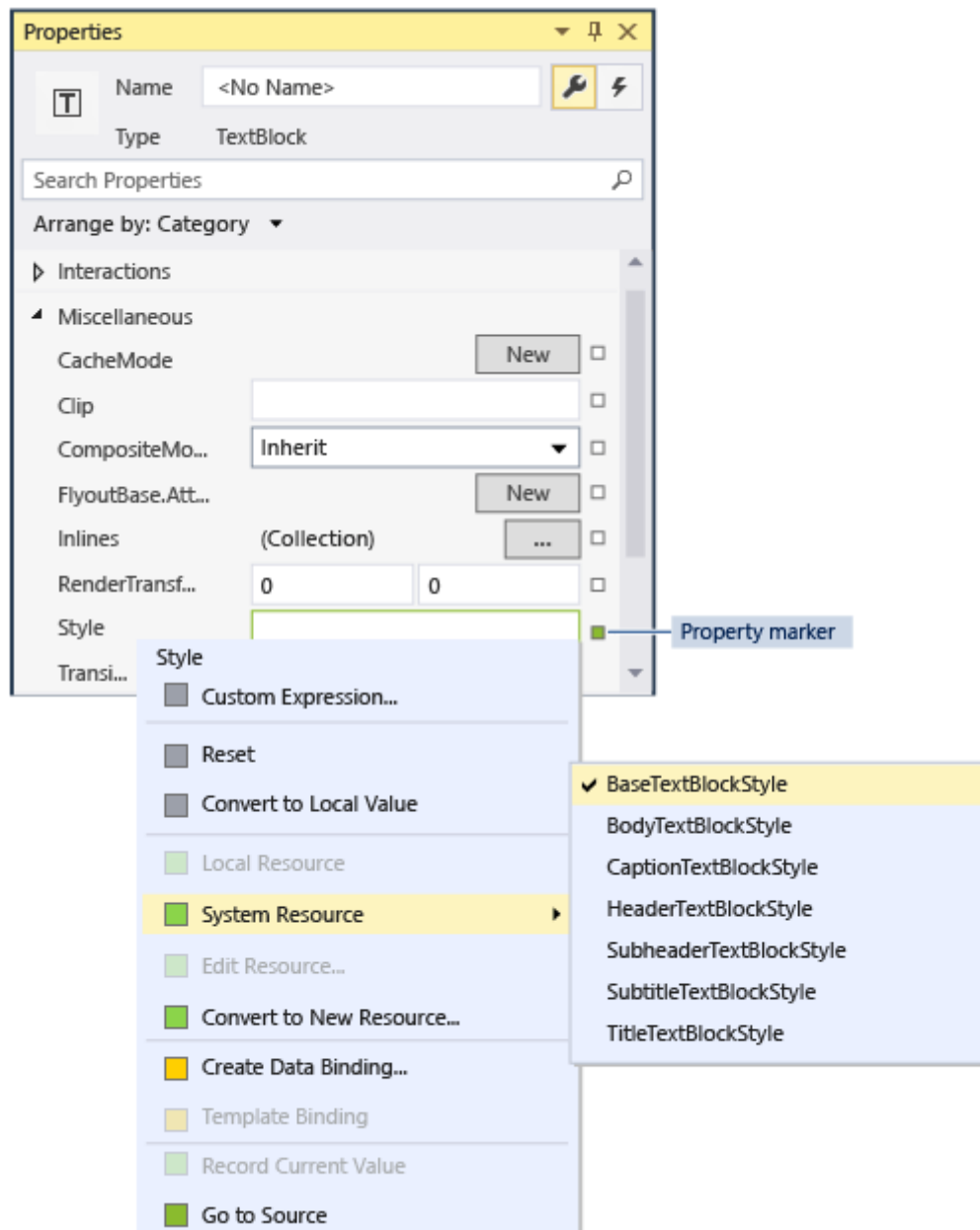
Right now, in the Windows app the text is very small and difficult to read. Let's fix that by applying a system style.

To change the style of an element

1. In the Windows project, open MainPage.xaml.

2. In either XAML or design view, select the "What's your name?" **TextBlock** that you added earlier.
3. In the **Properties** window (F4), choose the Properties button () in the upper right.
4. Expand the **Text** group and set the font size to 18 px.
5. Expand the **Miscellaneous** group and find the **Style** property.
6. Click the property marker (the green box to the right of the **Style** property), and then, on the menu, choose **System Resource** > **BaseTextBlockStyle**.

BaseTextBlockStyle is a resource that's defined in the **ResourceDictionary** in `<root>\Program Files\Windows Kits\10\Include\winrt\xaml\design\generic.xaml`.



On the XAML design surface, the appearance of the text changes. In the XAML editor, the XAML for the **TextBlock** is updated:

XAML

```
<TextBlock Text="What's your name?" Style="{ThemeResource BaseTextBlockStyle}"/>
```

7. Repeat the process to set the font size and assign the **BaseTextBlockStyle** to the `greetingOutput` **TextBlock** element.

Tip Although there's no text in this **TextBlock**, when you move the pointer over the XAML design surface, a blue outline shows where it is so that you can select it.

Your XAML now looks like this:

XAML

```
<StackPanel x:Name="contentPanel" Margin="120,30,0,0">
    <TextBlock Style="{ThemeResource BaseTextBlockStyle}" FontSize="18"
    Text="What's your name?"/>
    <StackPanel x:Name="inputPanel" Orientation="Horizontal"
    Margin="0,20,0,20">
        <TextBox x:Name="nameInput" Width="300" HorizontalAlignment="Left"/>
        <Button x:Name="inputButton" Content="Say "Hello""
        Click="Button_Click"/>
    </StackPanel>
    <TextBlock Style="{ThemeResource BaseTextBlockStyle}" FontSize="18"
    x:Name="greetingOutput"/>
</StackPanel>
```

8. Press F5 to build and run the app. It now looks like this:

Hello, world!

What's your name?

Say "Hello"

Hello, Somebody special!

Step 4: Adapt the UI to different window sizes

Now we'll make the UI adapt to different screen sizes so it looks good on mobile devices. To do this, you add a [VisualStateManager](#) and set properties that are applied for different visual states.

To adjust the UI layout

1. In the XAML editor, add this block of XAML after the opening tag of the root [Grid](#) element.

XAML

```
<VisualStateManager.VisualStateGroups>
    <VisualStateGroup>
        <VisualState x:Name="wideState">
            <VisualState.StateTriggers>
                <AdaptiveTrigger MinWindowWidth="641" />
            </VisualState.StateTriggers>
        </VisualState>
        <VisualState x:Name="narrowState">
            <VisualState.StateTriggers>
                <AdaptiveTrigger MinWindowWidth="0" />
            </VisualState.StateTriggers>
            <VisualState.Setters>
                <Setter Target="contentPanel.Margin" Value="20,30,0,0"/>
                <Setter Target="inputPanel.Orientation" Value="Vertical"/>
                <Setter Target="inputButton.Margin" Value="0,4,0,0"/>
            </VisualState.Setters>
        </VisualState>
    </VisualStateGroup>
</VisualStateManager.VisualStateGroups>
```

```
</VisualStateGroup>  
</VisualStateManager.VisualStateGroups>
```

2. Debug the app on the local machine. Notice that the UI looks the same as before unless the window gets narrower than 641 device-independent pixels (DIPs).
3. Debug the app on the mobile device emulator. Notice that the UI uses the properties you defined in the `narrowState` and appears correctly on the small screen.

Hello, world!

What's your name?

Say "Hello"

Hello, Somebody special!



If you've used a [VisualStateManager](#) in previous versions of XAML, you might notice that the XAML here uses a simplified syntax.

The [VisualState](#) named `wideState` has an [AdaptiveTrigger](#) with its [MinWindowWidth](#) property set to 641. This means that the state is to be applied only when the window width is not less than the minimum of 641 DIPs. You don't define any [Setter](#) objects for this state, so it uses the layout properties you defined in the XAML for the page content.

The second [VisualState](#), `narrowState`, has an [AdaptiveTrigger](#) with its [MinWindowWidth](#) property set to 0. This state is applied when the window width is greater than 0, but less than 641 DIPs. (At 641 DIPs, the `wideState` is applied.) In this state, you do define some [Setter](#) objects to change the layout properties of controls in the UI:

- You reduce the left margin of the `contentPanel1` element from 120 to 20.
- You change the **Orientation** of the `inputPanel1` element from **Horizontal** to **Vertical**.
- You add a top margin of 4 DIPs to the `inputButton` element.

Summary

Congratulations, you've completed the first tutorial! It taught how to add content to Windows Universal apps, how to add interactivity to them, and how to change their appearance.

Next steps

If you have a Windows Universal app project that targets Windows 8.1 and/or Windows Phone 8.1, you can port it to Windows 10 or Windows 11. There is no automatic process for this, but you can do it manually. Start with a new Windows Universal project to get the latest project system structure and manifest files, copy your code files into the project's directory structure, add the items to your project, and rewrite your XAML using the **VisualStateManager** according to the guidance in this topic. For more information, see [Porting a Windows Runtime 8 project to a Universal Windows Platform \(UWP\) project](#) and [Porting to the Universal Windows Platform \(C++\)](#).

If you have existing C++ code that you want to integrate with a UWP app, such as to create a new UWP UI for an existing application, see [How to: Use existing C++ code in a Universal Windows project](#).

Building a complex Universal Windows Platform (UWP) app

Article • 10/20/2022

On Microsoft design teams, our process for creating apps consists of five distinct stages: concept, structure, dynamics, visual, and prototype. We encourage you to adopt a similar process and have fun making new experiences for the world to enjoy.

ⓘ Note

It is recommended that developers who are creating new projects consider using the [Windows App SDK](#) and [WinUI](#) rather than UWP. Please see [Migrate from UWP to the Windows App SDK](#) for more details.

Concept

Focus your app

When planning your Universal Windows Platform (UWP) app, you should determine not only what your app will do and who it's for, but also what your app will be great at. At the core of every great app is a strong concept that provides a solid foundation.

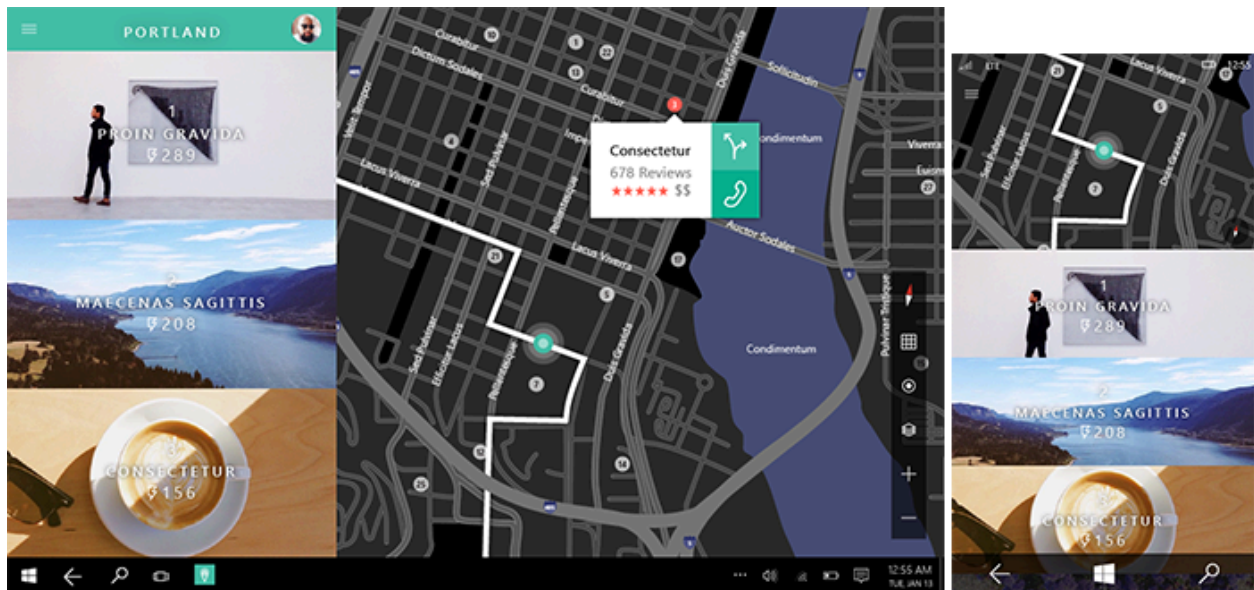
Say you want to create a photo app. Thinking about the reasons users work with, save, and share their photos, you'll realize that they want to relive memories, connect with others through the photos, and keep the photos safe. These, then, are the things that you want the app to be great at, and you use these experience goals to guide you through the rest of the design process.

What's your app about? Start with a broad concept and list all of the things that you want to help users do with your app.

For example, suppose you want to build an app that helps people plan their trips. Here are some ideas you might sketch out on the back of a napkin:

- Get maps of all the places on an itinerary, and take them with you on the trip.
- Find out about special events happening while you're in a city.
- Let travel buddies create separate but shareable lists of must-do activities and must-see attractions.
- Let travel buddies compile all of their photos to share with friends and family.
- Get recommended destinations based on flight prices.

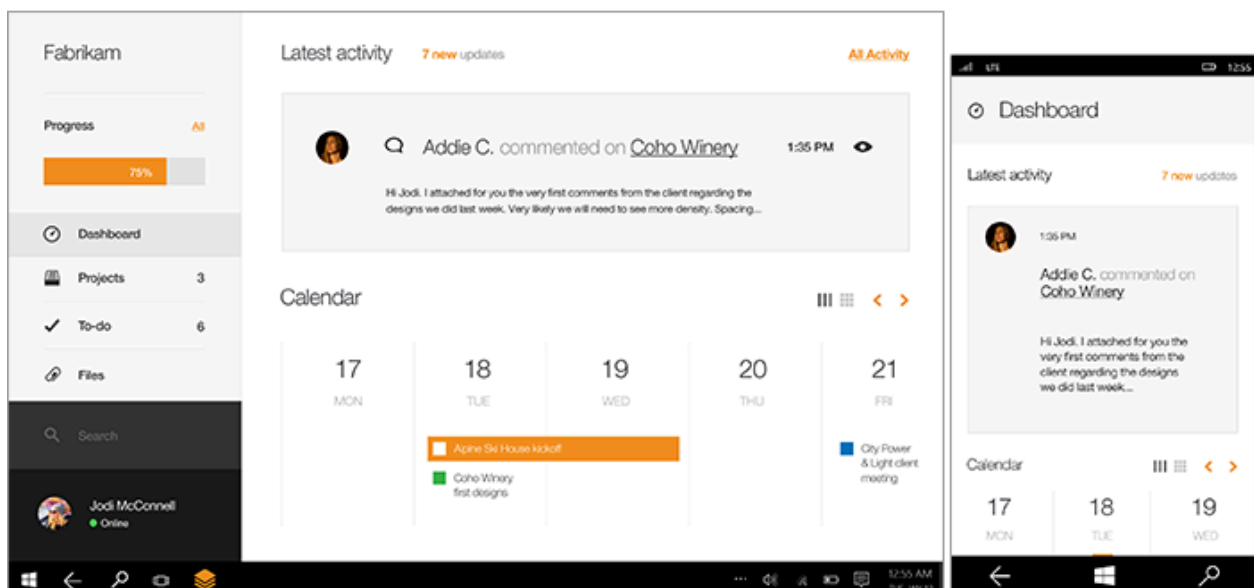
- Find a consolidated list of deals for restaurants, shops, and activities around your destination.



What's your app great at? Take a step back and look at your list of ideas to see if a particular scenario really jumps out at you. Challenge yourself to trim the list to just a single scenario that you want to focus on. In the process, you might cross off many good ideas, but saying "no" to them is crucial to making a single scenario great.

After you choose a single scenario, decide how you would explain to an average person what your app is great at by writing it down in one sentence. For example:

- My travel app is great at helping friends create itineraries collaboratively for group trips.
- My workout app is great at letting friends track their workout progress and share their achievements with each other.
- My grocery app is great at helping families coordinate their weekly grocery shopping so they never miss or duplicate a purchase.



This is your app's "great at" statement, and it can guide many design decisions and tradeoffs that you make as you build your app. Focus on the scenarios you want users to experience in your app, and be careful not to turn this into a feature list. It should be about what your users will be able to do, as opposed to what your app will be able to do.

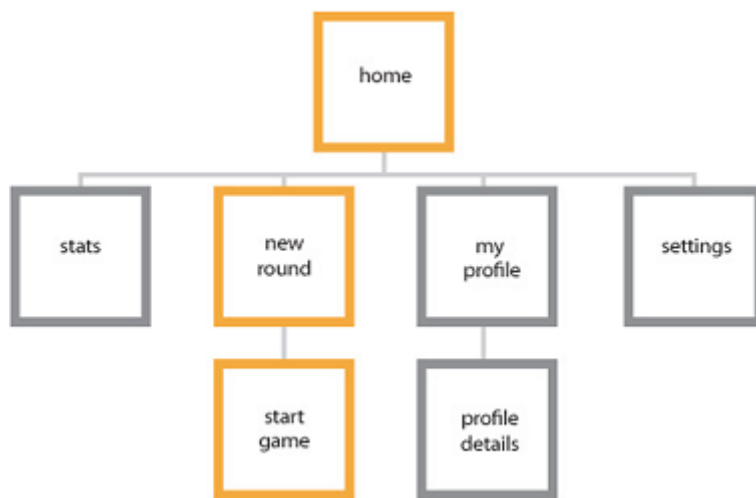
The design funnel

It's very tempting—having thought of an idea you like—to go ahead and develop it, perhaps even taking it quite a ways into production. But let's say you do that and then another interesting idea comes along. It's natural that you'll be tempted to stick with the idea you've already invested in regardless of the relative merits of the two ideas. If only you'd thought of that other idea earlier in the process! Well, the design funnel is a technique to help uncover your best ideas as early as possible.

The term "funnel" comes from its shape. At the wide end of the funnel, many ideas go in and each one is realized as a very low-fidelity design artifact (a sketch, perhaps, or a paragraph of text). As this collection of ideas travels through toward the narrow end of the funnel, the number of ideas is trimmed down while the fidelity of the artifacts representing the ideas increases. Each artifact should capture only the information necessary to judge one idea against another, or to answer a particular question such as "is this usable, or intuitive?". *Put no more time and effort into each than that.* Some ideas will fall by the wayside as you test them, and you'll be okay with that because you won't be invested in them any more than was necessary to judge the idea. Ideas that survive to move further into the funnel will receive successively high-fidelity treatments. In the end, you'll have a single design artifact that represents the winning idea. This is the idea that won because of its merits, not merely because it came along first. You will have designed the best app you could.

Structure

Organization makes everything easier



When you're happy with your concept, you're ready for the next stage—creating your app's blueprint. Information architecture (IA) gives your content the structural integrity it needs. It helps define your app's navigational model and, ultimately, your app's identity. By planning how your content will be organized—and how your users will discover that content—you can get a better idea of how users will experience your app.

Good IA not only facilitates user scenarios, but it helps you envision the key screens to start with. The [Audible](#) app, for example, launches directly into a hub that provides access to the user's library, store, news, and stats. The experience is focused, so users can get and enjoy audiobooks quickly. Deeper levels of the app focus on more specific tasks.

For related guidelines, see [Navigation design basics](#).

Dynamics

Execute your concept

If the concept stage is about defining your app's purpose, the dynamics stage is all about executing that purpose. This can be accomplished in many ways, such as using wireframes to sketch out your page flows (how you get from one place to the next within the app to achieve their goals), and thinking about the voice and the words used throughout your app's UI. Wireframes are a quick, low-fidelity tool to help you make critical decisions about your app's user flow.

Your app flow should be tightly tied to your "great at" statement, and should help users achieve that single scenario that you want to light up. Great apps have flows that are easy to learn, and require minimal effort. Start thinking on a screen-to-screen level—see your app as if you're using it for the first time. When you pinpoint user scenarios for pages you create, you'll give people exactly what they want without lots of unnecessary

screen touches. Dynamics are also about motion. The right motion capabilities will determine fluidity and ease of use from one page to the next.

Common techniques to help with this step:

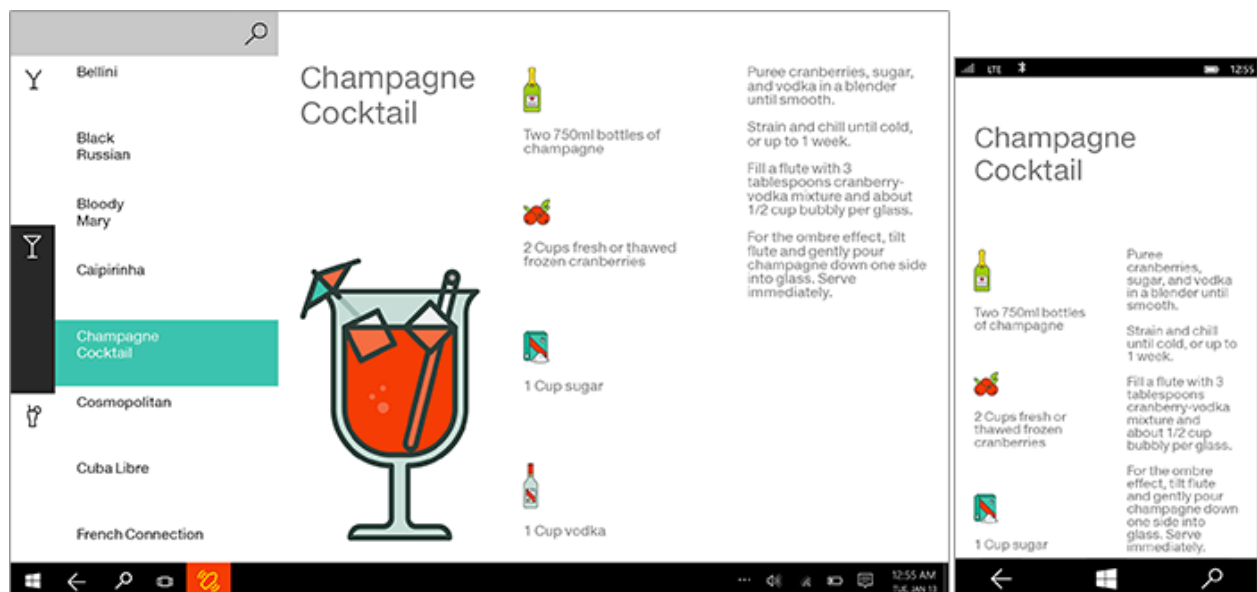
- Outline the flow: What comes first, what comes next?
- Storyboard the flow: How should users move through your UI to complete the flow?
- Prototype: Try out the flow with a quick prototype.

What should users be able to do? For example, the travel app is "great at helping friends collaboratively create itineraries for group trips." Let's list the flows that we want to enable:

- Create a trip with general information.
- Invite friends to join a trip.
- Join a friend's trip.
- See itineraries recommended by other travelers.
- Add destinations and activities to trips.
- Edit and comment on destinations and activities that friends added.
- Share itineraries for friends and families to follow.

Visual

Speak without words



Once you've established the dynamics of your app, you can make your app shine with the right visual polish. Great visuals define not only how your app looks, but how it feels and comes alive through animation and motion. Your choice of color palette, icon, and artwork are just a few examples of this visual language.

All apps have their own unique identity, so explore the visual directions you can take with your app. Let the content guide the look and feel; don't let the look dictate your content.

Prototype

Refine your masterpiece

Prototyping is a stage in the *design funnel*—a technique we talked about earlier—at which the artifact representing your idea develops into something more than a sketch, but less complicated than a complete app. A prototype might be a flow of hand-drawn screens shown to a user. The person running the test might respond to cues from the user by placing different screens down, or sticking or unsticking smaller pieces of UI on the pages, to simulate a running app. Or, a prototype might be a very simple app that simulates some workflows, provided the operator sticks to a script and pushes the right buttons. At this stage, your ideas begin to really come alive and your hard work is tested in earnest. When prototyping areas of your app, take the time to sculpt and refine the components that need it the most.

To new developers, we can't stress enough: Making great apps is an iterative process. We recommend that you prototype early and often. Like any creative endeavor, the best apps are the product of intensive trial and error.

Decide what features to include

When you know what your users want and how you can help them get there, you can look at the specific tools in your toolbox. Explore the Universal Windows Platform (UWP) and associate features with your app's needs. Be sure to follow the [user experience \(UX\) guidelines](#) for each feature.

Common techniques:

- Platform research: Find out what features the platform offers and how you can use them.
- Association diagrams: Connect your flows with features.
- Prototype: Exercise the features to ensure that they do what you need.

App contracts Your app can participate in app contracts that enable broad, cross-app, cross-feature user flows.

- **Share** Let your users share content from your app with other people through other apps, and receive shareable content from other people and apps, too.

- **Play To** Let your users enjoy audio, video, or images streamed from your app to other devices in their home network.
- **File picker and file picker extensions** Let your users load and save their files from the local file system, connected storage devices, HomeGroup, or even other apps. You can also provide a file picker extension so other apps can load your app's content.

For more info, see [App contracts and extensions](#).

Different views, form factors, and hardware configurations Windows puts users in charge and your app in the forefront. You want your app UI to shine on any device, using any input mode, in any orientation, in any hardware configuration, and in whatever circumstance the user decides to use it.

Touch first Windows provides a unique and distinctive touch experience that does more than simply emulate mouse functionality.

For example, semantic zoom is a touch-optimized way to navigate through a large set of content. Users can pan or scroll through categories of content, and then zoom in on those categories to view more and more detailed information. You can use this to present your content in a more tactile, visual, and informative way than with traditional navigation and layout patterns like tabs.

Of course, you can take advantage of a number of touch interactions, like rotate, pan, swipe, and others. Learn more about [Touch and other user interactions](#).

Engaging and fresh Be sure your app feels fresh and engages users with these standard experiences:

- **Animations** Use our library of animations to make your app fast and fluid for your users. Help users understand context changes and tie experiences together with visual transitions. Learn more about [animating your UI](#).
- **Toast notifications** Let your users know about time-sensitive or personally relevant content through toast notifications, and invite them back to your app even when your app is closed. Learn more about [tiles, badges, and toast notifications](#).
- **App tiles** Provide fresh and relevant updates to entice users back into your app. There's more info about this in the next section. Learn more about [app tiles](#).

Personalization

- **Settings** Let your users create the experience they want by saving app settings. Consolidate all of your settings on one screen, and then users can configure your

app through a common mechanism that they are already familiar with. Learn more about [Adding app settings](#).

- **User tiles** Make your app more personal to your users by loading their user tile image, or let the users set content from your app as their personal tile throughout Windows.

Device capabilities Be sure your app takes full advantage of the capabilities of today's devices.

- **Proximity gestures** Let your users connect devices with other users who are physically in close proximity, by physically tapping the devices together (multiplayer games). Learn more about [proximity and tapping](#).
- **Cameras and external storage devices** Connect your users to their built-in or plugged-in cameras for chatting and conferencing, recording vlogs, taking profile pics, documenting the world around them, or whatever activity your app is great at. Learn more about [Accessing content on removable storage](#).
- **Accelerometers and other sensors** Devices come with a number of sensors nowadays. Your app can dim or brighten the display based on ambient light, reflow the UI if the user rotates the display, or react to any physical movement. Learn more about [sensors](#).
- **Geolocation** Use geolocation information from standard web data or from geolocation sensors to help your users get around, find their position on a map, or get notices about nearby people, activities, and destinations. Learn more about [geolocation](#).

Let's consider the travel app example again. To be great at helping friends collaboratively create itineraries for group trips, you could use some of these features, just to name a few:

- **Share:** Users share upcoming trips and their itineraries to multiple social networks to share the pre-trip excitement with their friends and families.
- **Search:** Users search for and find activities or destinations from others' shared or public itineraries that they can include in their own trips.
- **Notifications:** Users are notified when travel companions update their itineraries.
- **Settings:** Users configure the app to their preference, like which trip should bring up notifications or which social groups are allowed to search the users' itineraries.
- **Semantic zoom:** Users navigate through the timeline of their itinerary and zoom in to see greater details of the long list of activities they've planned.
- **User tiles:** Users choose the picture they want to appear when they share their trip with friends.

Decide how to monetize your app

You have a lot of options for earning money from your app. If you decide to use in-app ads or sales, you'll want to design your UI to support that. For more information, see [Plan for monetization](#).

Design the UX for your app

This is about getting the basics right. Now that you know what your app is great at, and you've figured out the flows that you want to support, you can start to think about the fundamentals of user experience (UX) design.

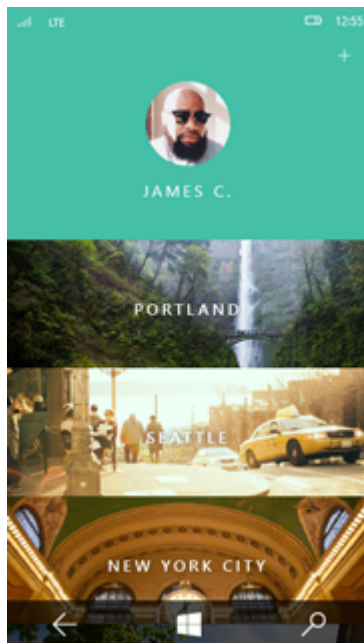
How should you organize UI content? Most app content can be organized into some form of groupings or hierarchies. What you choose as the top-level grouping of your content should match the focus of your "great at" statement.

To use the travel app as an example, there are multiple ways to group itineraries. If the focus of the app is discovering interesting destinations, you might group them based on interest, like adventure, fun in the sun, or romantic getaways. However, because the focus of the app is planning trips with friends, it makes more sense to organize itineraries based on social circles, like family, friends, or work.

Choosing how you want to group your content helps you decide what pages or views you need in your app. See [UI basics](#) for more info.

How should you present UI content? After you've decided how to organize your UI, you can define UX goals that specify how your UI gets built and presented to your user. In any scenario, you want to make sure that your user can continue using and enjoying your app as quickly as possible. To do this, decide what parts of your UI need to be presented first, and make sure that those parts are complete before you spend time building the noncritical parts.

In the travel app, probably the first thing the user will want to do in the app is find a specific trip itinerary. To present this info as fast as possible, you should show the list of trips first, using a **ListView** control.



After showing the trips list, you could start loading other features, like a news feed of their friends' trips.

What UI surfaces and commands do you need? Review the flows that you identified earlier. For each flow, create a rough outline of the steps users take.

Let's look at the "Share itineraries for friends and families to follow" flow. We'll assume that the user has already created a trip. Sharing a trip itinerary might require these steps:

1. The user opens the app and sees a list of trips she created.
2. The user taps on the trip she wants to share.
3. The details of the trip appear on screen.
4. The user accesses some UI to initiate sharing.
5. The user selects or enters the email address or name of the friend she wants to share the trip with.
6. The user accesses some UI to finalize sharing.
7. Your app updates the trip details with the list of people she has shared her trip with.

During this process, you begin to see what UI you need to create and the additional details you need to figure out (like drafting a standard email boilerplate for friends who aren't using your app yet). You also can start eliminating unnecessary steps. Perhaps the user doesn't actually need to see the details of the trip before sharing, for example. The cleaner the flow, the easier to use.

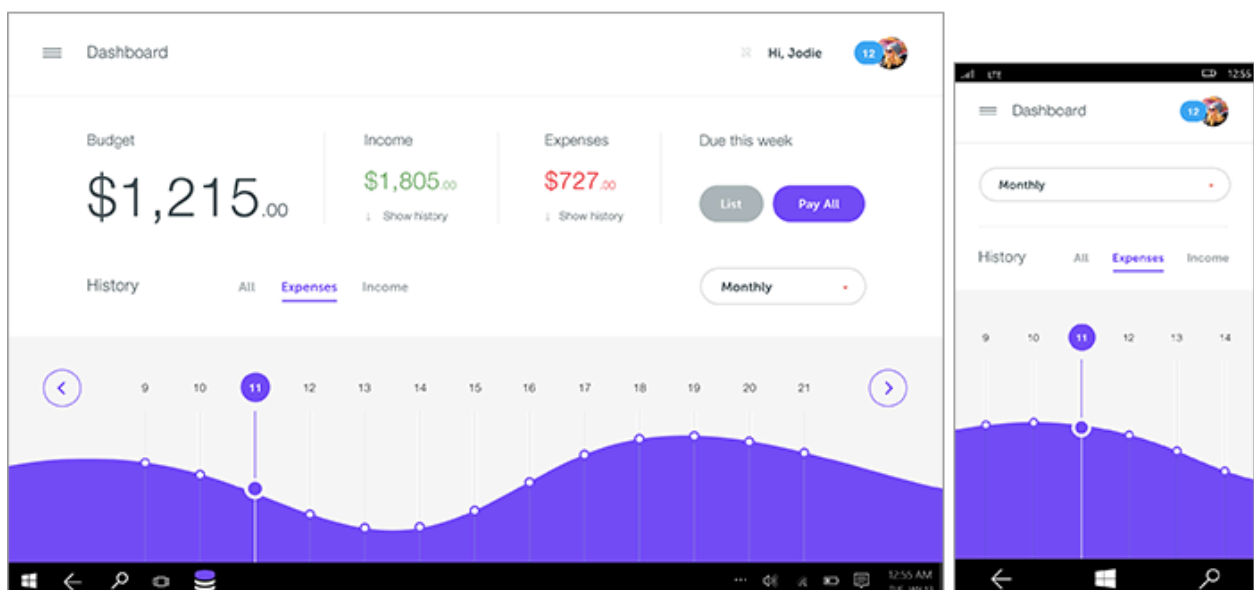
For more details on how to use different surfaces, take a look at .

What should the flow feel like? When you have defined the steps your user will take, you can turn that flow into performance goals. For more info, see [Plan for performance](#).

How should you organize commands? Use your outline of the flow steps to identify potential commands that you need to design for. Then think about where to use those commands in your app.

- **Always try to use the content.** Whenever possible, let users directly manipulate the content on the app's canvas, rather than adding commands that act on the content. For example, in the travel app, let users rearrange their itinerary by dragging and dropping activities in a list on the canvas, rather than by selecting the activity and using Up or Down command buttons.
- **If you can't use the content.** Place commands on one of these UI surfaces if you are not able to use the content:
 - In the **command bar**: You should put most commands on the command bar, which is usually hidden until the user taps to make it visible.
 - On the app's canvas: If the user is on a page or view that has a single purpose, you can provide commands for that purpose directly on the canvas. There should be very few of these commands.
 - In a **context menu**: You can use context menus for clipboard actions (such as cut, copy, and paste), or for commands that apply to content that cannot be selected (like adding a push pin to a location on a map).

Decide how to lay out your app in each view. Windows supports landscape and portrait orientations and supports resizing apps to any width, from full screen to a minimum width. You want your app to look and work great at any size, on any screen, in either orientation. This means you need to plan the layout of your UI elements for different sizes and views. When you do this, your app UI changes fluidly to meet your user's needs and preferences.



For more info on designing for different screen sizes, see [Screen sizes and break points for responsive design](#).

Make a good first impression

Think about what you want users to think, feel, or do when they first launch your app. Refer back to your "great at" statement. Even though you won't get a chance to personally tell your users what your app is great at, you can convey the message to them when you make your first impression. Take advantage of these:

Tile and notifications The tile is the face of your app. Among the many other apps on a user's Start screen, what will make the user want to launch your app? Be sure your tile highlights your app's brand and shows what the app is great at. Use tile notifications so your app will always feel fresh and relevant, bringing the user back to your app again and again.

Splash screen The splash screen should load as fast as possible, and remain on the screen only as long as you need to initialize your app state. What you show on the splash screen should express your app's personality.

First launch Before users sign up for your service, log in to their account, or add their own content, what will they see? Try to demonstrate the value of your app before asking users for information. Consider showing sample content so people can look around and understand what your app does before you ask them to commit.

Home page The home page is where you bring users each time they launch your app. The content here should have a clear focus, and immediately showcase what your app is tailored to do. Make this page great at one thing and trust that people will explore the rest of your app. Focus on eliminating distractions on the landing page, and not on discoverability.

Validate your design

Before you get too far into developing your app, you should validate your design or prototype against guidelines, user impressions, and requirements to avoid having to rework it later. Each feature has a set of UX guidelines to help you polish your app, and a set of Store requirements that you must meet to publish your app in the Microsoft Store. You can use [the Windows App Certification Kit](#) to test for technical compliance with Store requirements. You can also use the performance tools in Microsoft Visual Studio to make sure that you're giving your users a great experience in every scenario.

Use the [detailed UX guidelines for UWP apps](#) to stay focused on important features. Use the [Visual Studio performance tools](#) to analyze the performance of each of your app's scenarios.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Develop UWP apps

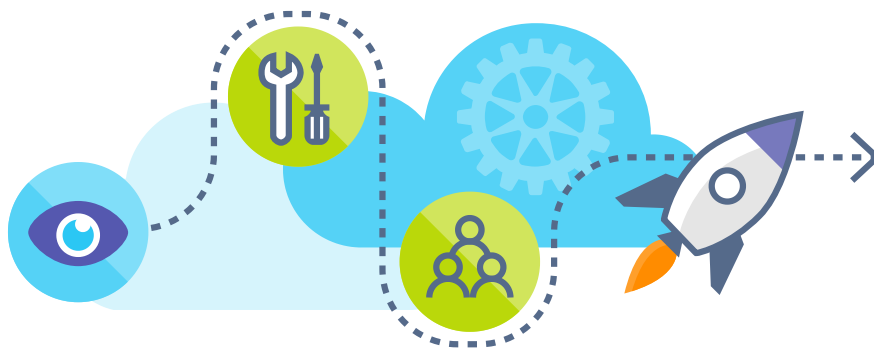
Article • 03/15/2023

How-to articles and code for creating UWP apps for Windows.



Overview of the Universal Windows Platform

An explanation of what UWP is, how it works, and the features it provides.



Porting guide

Bring your existing Windows Forms, WPF, Android, or iOS app to UWP.

API reference

Windows UWP namespaces

The classes, structures, interfaces, methods, properties, and events that make up the Windows Runtime, organized by namespace.

Schemas for UWP

File and XML schema specifications for Universal Windows Platform (UWP) apps.

Articles

App types

[Education apps](#)

[Enterprise apps](#)

[Games and DirectX apps](#)

[Progressive Web Apps](#)

App UI

For controls, layout, typography, animation, usability, and UI design, see the [Design and UI](#) section.

Communication

[App-to-app communication](#)

[Networking and web services](#)

Data and files

[Audio, video, and camera](#)

[Data access](#)

[Data binding](#)

[Files, folders, and libraries](#)

[Machine learning](#)

Deployment

[Choose a UWP version](#)

[Debugging, testing, and performance](#)

[Monetization, engagement, and Store services](#)

[Packaging apps](#)

Platform

[C++/WinRT](#)

[Launching, resuming, and background tasks](#)

[Security](#)

[Threading and async programming](#)

[Visual layer](#)

[Windows as a service](#)

[Windows Runtime components](#)

[XAML platform](#)

People and places

[Contacts, My People, and calendar](#)

[Maps and location](#)

Peripherals, sensors, and power

[Overview](#)

[Enable device capabilities](#)

[Pair devices](#)

[Point of Service](#)

[Sensors](#)

[Printing](#)

[3-D printing<](#)

[NFC](#)

[Battery info](#)

Porting

[Overview](#)

[Windows Phone Silverlight to UWP](#)

[Windows Runtime 8.x to UWP](#)

[Desktop Bridge](#)

[Share code between desktop and UWP](#)

[Concept mapping for Android and iOS developers](#)

[Move from iOS to UWP](#)

[Convert your web app to a PWA](#)

[Windows on ARM](#)

Processes and threading

Launching, resuming, and background tasks

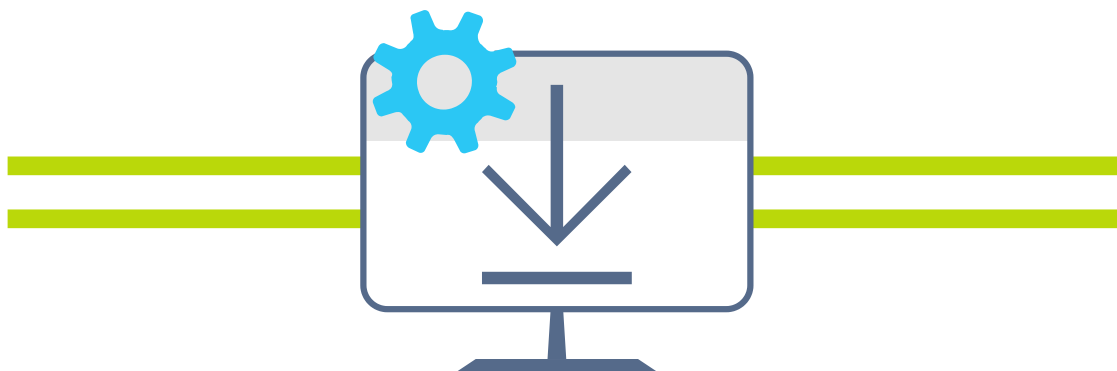
Threading and async programming

Samples and tools



Samples

Learn how to build great apps for Windows by experimenting with these samples. These samples show you how features work and help you jumpstart your own UWP apps.

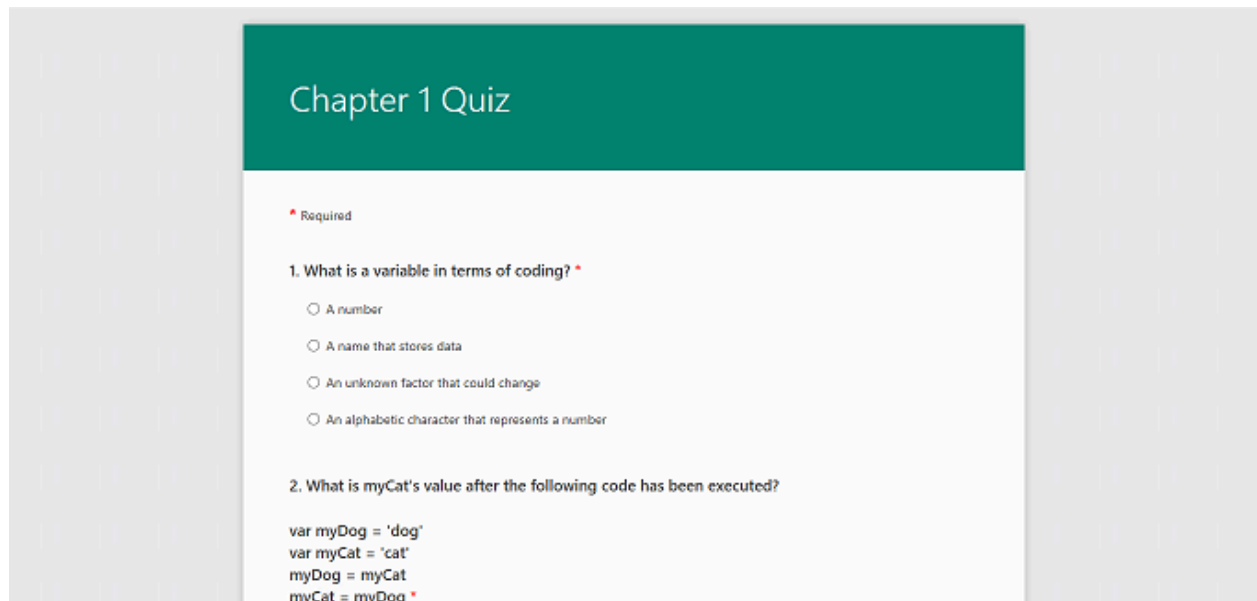


Developer tools

Get Visual Studio 2019, the Windows SDK, and other developer tools.

Develop Universal Windows apps for education

Article • 11/01/2022



The following resources will help you write a Universal Windows app for education.

Accessibility

Education apps need to be accessible. See [Developing apps for accessibility](#) for more information.

Secure assessments

Assessment/testing apps will often need to produce a *locked down* environment in order to prevent students from using other computers or Internet resources during a test. This functionality is available through the [Take a Test API](#). See the [Take a Test](#) web app in the Windows IT Center for an example of a testing environment with locked down online access for high-stakes testing.

User input

User input is a critical part of education apps; UI controls must be responsive and intuitive so as not to break the focus of their users. For a general overview of the input options available in a Universal Windows app, see the [Input primer](#) and the topics below

it in the Design & UI section. Additionally, the following sample apps showcase basic UI handling in the Universal Windows Platform.

- [Basic input sample](#)  shows how to handle input in Universal Windows Apps.
- [User interaction mode sample](#)  shows how to detect and respond to the user interaction mode.
- [Focus visuals sample](#)  shows how to take advantage of the new system drawn focus visuals or create your own custom focus visuals if the system drawn ones do not fit your needs.

The Windows Ink platform can make education apps shine by fitting them with an input mode that students are accustomed to. See [Pen interactions and Windows Ink](#) and the topics below it for a comprehensive guide to implementing Windows Ink in your app. The following sample apps provide working examples of this API.

- [Ink Analysis sample](#)  demonstrates how to use ink functionality (such as capturing, manipulating, and interpreting ink strokes) in Universal Windows apps using JavaScript.
- [Simple ink sample](#)  demonstrates how to use ink functionality (such as capturing ink from user input and performing handwriting recognition on ink strokes) in Universal Windows apps using C#.
- [Complex ink sample](#)  demonstrates how to use advanced InkPresenter functionality to interleave ink with other objects, select ink, copy/paste, and handle events. It is built upon the Universal Windows Platform in C++ and can run on Desktop and Mobile Windows 10 and Windows 11 SKUs.

Microsoft Store

Education apps are often released under special circumstances to a specific organization. See [Distribute line-of-business apps to enterprises](#) for information on this.

Related Topics

- [Windows 10/11 for Education](#) on the Windows IT Center

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Take a Test JavaScript API

Article • 11/15/2023

[Take a Test](#) is a browser-based UWP app that renders locked-down online assessments for high-stakes testing, allowing educators to focus on the assessment content rather than how to provide a secure testing environment. To achieve this, it uses a JavaScript API that any web application can utilize. The Take-a-test API supports the [SBAC browser API standard](#) [↗](#) for high stakes common core testing.

See the [Take a Test app technical reference](#) for more information about the app itself. For troubleshooting help, see [Troubleshoot Microsoft Take a Test with the event viewer](#).

Reference documentation

The Take a Test APIs exist in the following namespaces. Note that all of the APIs depend on a global `SecureBrowser` object.

Namespace	Description
security namespace	Contains APIs that enable you to lock down the device for testing and enforce a testing environment.

Security namespace

The security namespace allows you to lock down the device, check the list of user and system processes, obtain MAC and IP addresses, and clear cached web resources.

Method	Description
lockDown	Locks down the device for testing.
isEnvironmentSecure	Determines whether the lockdown context is still applied to the device.
getDeviceInfo	Gets details about the platform on which the testing application is running.
examineProcessList	Gets the list of running user and system processes.
close	Closes the browser and unlocks the device.
getPermissiveMode	Checks if permissive mode is on or off.
setPermissiveMode	Toggles permissive mode on or off.

Method	Description
<code>emptyClipboard</code>	Clears the system clipboard.
<code>getMACAddress</code>	Gets the list of MAC addresses for the device.
<code>getStartTime</code>	Gets the time that the testing app was started.
<code>getCapability</code>	Queries whether a capability is enabled or disabled.
<code>setCapability</code>	Enables or disables the specified capability.
<code>isRemoteSession</code>	Checks if the current session is logged in remotely.
<code>isVMSession</code>	Checks if the current session is running in a virtual machine.

lockDown

Locks down the device. Also used to unlock the device. The testing web application will invoke this call prior to allowing students to start testing. The implementer is required to take any actions necessary to secure the testing environment. The steps taken to secure the environment are device specific and for example, include aspects such as disabling screen captures, disabling voice chat when in secure mode, clearing the system clipboard, entering into a kiosk mode, disabling Spaces in OSX 10.7+ devices, etc. The testing application will enable lockdown before an assessment commences and will disable the lockdown when the student has completed the assessment and is out of the secure test.

Syntax

```
void SecureBrowser.security.lockDown(Boolean enable, Function onSuccess, Function onError);
```

Parameters

- `enable` - **true** to run the Take-a-Test app above the lock screen and apply policies discussed in this [document](#). **false** stops running Take-a-Test above the lock screen and closes it unless the app is not locked down; in which case there is no effect.
- `onSuccess` - [optional] The function to call after the lockdown has been successfully enabled or disabled. It must be of the form `Function(Boolean currentlockdownstate)`.
- `onError` - [optional] The function to call if the lockdown operation failed. It must be of the form `Function(Boolean currentlockdownstate)`.

Requirements

Windows 10, version 1709 or later

isEnvironmentSecure

Determines whether the lockdown context is still applied to the device. The testing web application will invoke this prior to allowing students to start testing and periodically when inside the test.

Syntax

```
void SecureBrowser.security.isEnvironmentSecure(Function callback);
```

Parameters

- `callback` - The function to call when this function completes. It must be of the form `Function(String state)` where `state` is a JSON string containing two fields. The first is the `secure` field, which will show `true` only if all necessary locks have been enabled (or features disabled) to enable a secure testing environment, and none of these have been compromised since the app entered the lockdown mode. The other field, `messageKey`, includes other details or information that is vendor-specific. The intent here is to allow vendors to put additional information that augments the boolean `secure` flag:

JSON

```
{
  'secure' : "true/false",
  'messageKey' : "some message"
}
```

Requirements

Windows 10, version 1709 or newer

getDeviceInfo

Gets details about the platform on which the testing application is running. This is used to augment any information that was discernible from the user agent.

Syntax

```
void SecureBrowser.security.getDeviceInfo(Function callback);
```

Parameters

- `callback` - The function to call when this function completes. It must be of the form `Function(String infoObj)` where `infoObj` is a JSON string containing several fields. The following fields must be supported:
 - `os` represents the OS type (for example: Windows, macOS, Linux, iOS, Android, etc.)
 - `name` represents the OS release name, if any (for example: Sierra, Ubuntu).
 - `version` represents the OS version (for example: 10.1, 10 Pro, etc.)
 - `brand` represents the secure browser branding (for example: OAKS, CA, SmarterApp, etc.)
 - `model` represents the device model for mobile devices only; null/unused for desktop browsers.

Requirements

Windows 10, version 1709 or newer

examineProcessList

Gets the list of all processes running on the client machine owned by the user. The testing application will invoke this to examine the list and compare it with a list of processes that have been deemed deny-listed during testing cycle. This call should be invoked both at the start of an assessment and periodically while the student is taking the assessment. If a deny-listed process is detected, the assessment should be stopped to preserve test integrity.

Syntax

```
void SecureBrowser.security.examineProcessList(String[] denylistedProcessList,
Function callback);
```

Parameters

- `denylistedProcessList` - The list of processes that the testing application has deny-listed.
- `callback` - The function to invoke once the active processes have been found. It must be in the form: `Function(String foundDenylistedProcesses)` where `foundDenylistedProcesses` is in the form: `"['process1.exe', 'process2.exe', 'processEtc.exe']"`. It will be empty if no deny-listed processes were found. If it is null, this indicates that an error occurred in the original function call.

Remarks The list does not include system processes.

Requirements

Windows 10, version 1709 or newer

close

Closes the browser and unlocks the device. The testing application should invoke this when the user elects to exit the browser.

Syntax

```
void SecureBrowser.security.close(restart);
```

Parameters

- `restart` - This parameter is ignored but must be provided.

Remarks In Windows 10, version 1607, the device must be locked down initially. In later versions, this method closes the browser regardless of whether the device is locked down.

Requirements

Windows 10, version 1709 or newer

getPermissiveMode

The testing web application should invoke this to determine if permissive mode is on or off. In permissive mode, a browser is expected to relax some of its stringent security hooks to allow assistive technology to work with the secure browser. For example, browsers that aggressively prevent other application UIs from presenting on top of them might want to relax this when in permissive mode.

Syntax

```
void SecureBrowser.security.getPermissiveMode(Function callback)
```

Parameters

- `callback` - The function to invoke when this call completes. It must be in the form: `Function(Boolean permissiveMode)` where `permissiveMode` indicates whether the browser is currently in permissive mode. If it is undefined or null, an error occurred in the get operation.

Requirements

Windows 10, version 1709 or newer

setPermissiveMode

The testing web application should invoke this to toggle permissive mode on or off. In permissive mode, a browser is expected to relax some of its stringent security hooks to allow assistive technology to work with the secure browser. For example, browsers that aggressively prevent other application UIs from presenting on top of them might want to relax this when in permissive mode.

Syntax

```
void SecureBrowser.security.setPermissiveMode(Boolean enable, Function callback)
```

Parameters

- `enable` - The Boolean value indicating the intended permissive mode status.
- `callback` - The function to invoke when this call completes. It must be in the form: `Function(Boolean permissiveMode)` where `permissiveMode` indicates whether the browser is currently in permissive mode. If it is undefined or null, an error occurred in the set operation.

Requirements

Windows 10, version 1709 or newer

emptyClipboard

Clears the system clipboard. The testing application should invoke this to force clear any data that may be stored in the system clipboard. The [lockDown](#) function also performs this operation.

Syntax

```
void SecureBrowser.security.emptyClipboard();
```

Requirements

Windows 10, version 1709 or newer

getMACAddress

Gets the list of MAC addresses for the device. The testing application should invoke this to assist in diagnostics.

Syntax

```
void SecureBrowser.security.getMACAddress(Function callback);
```

Parameters

- `callback` - The function to invoke when this call completes. It must be in the form: `Function(String addressArray)` where `addressArray` is in the form: `"['00:11:22:33:44:55', 'etc']"`.

Remarks

It is difficult to rely on source IP addresses to distinguish between end user machines within the testing servers because firewalls/NATs/Proxies are commonly in use at the schools. The MAC addresses allow the app to distinguish end client machines behind a common firewall for diagnostics purposes.

Requirements

Windows 10, version 1709 or newer

getStartTime

Gets the time that the testing app was started.

Syntax

```
DateTime SecureBrowser.security.getStartTime();
```

Return

A `DateTime` object indicating the time the testing app was started.

Requirements

Windows 10, version 1709 or newer

getCapability

Queries whether a capability is enabled or disabled.

Syntax

```
Object SecureBrowser.security.getCapability(String feature)
```

Parameters

`feature` - The string to determine which capability to query. Valid capability strings are "screenMonitoring", "printing", and "textSuggestions" (case insensitive).

Return Value

This function returns either a JavaScript Object or literal with the form:

`{<feature>:true|false}`. **true** if the queried capability is enabled, **false** if the capability is not enabled or the capability string is invalid.

Requirements Windows 10, version 1703 or newer

setCapability

Enables or disables a specific capability on the browser.

Syntax

```
void SecureBrowser.security.setCapability(String feature, String value, Function onSuccess, Function onError)
```

Parameters

- **feature** - The string to determine which capability to set. Valid capability strings are "screenMonitoring", "printing", and "textSuggestions" (case insensitive).
- **value** - The intended setting for the feature. It must be either "true" or "false".
- **onSuccess** - [optional] The function to call after the set operation has been completed successfully. It must be of the form `Function(String jsonValue)` where *jsonValue* is in the form: `{<feature>:true|false|undefined}`.
- **onError** - [optional] The function to call if the set operation failed. It must be of the form `Function(String jsonValue)` where *jsonValue* is in the form: `{<feature>:true|false|undefined}`.

Remarks

If the targeted feature is unknown to the browser, this function will pass a value of `undefined` to the callback function.

Requirements Windows 10, version 1703 or newer

isRemoteSession

Checks if the current session is logged in remotely.

Syntax

```
Boolean SecureBrowser.security.isRemoteSession();
```

Return value

true if the current session is remote, otherwise **false**.

Requirements

Windows 10, version 1709 or later

isVMSession

Checks if the current session is running within a virtual machine.

Syntax

```
Boolean SecureBrowser.security.isVMSession();
```

Return value

true if the current session is running in a virtual machine, otherwise **false**.

Remarks

This API check can only detect VM sessions that are running in certain hypervisors that implement the appropriate APIs

Requirements

Windows 10, version 1709 or later

Enterprise

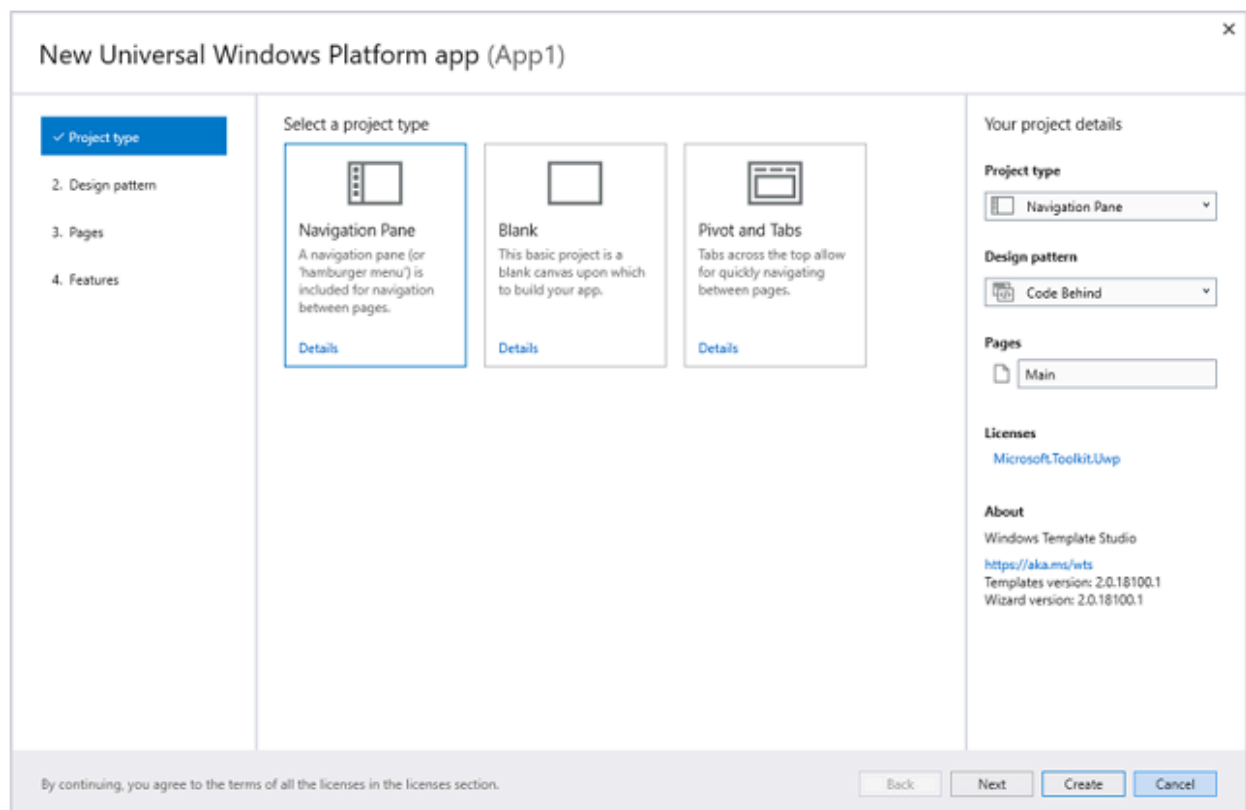
Article • 11/01/2022

This article provides an overview of key enterprise features provided by the Universal Windows Platform (UWP) for Windows apps. For a video that demonstrates some of these features in detail, see [Rapidly Construct LOB Applications with UWP and Visual Studio](#).

Feature highlights

Windows Template Studio

Windows Template Studio is a Visual Studio 2019 extension that accelerates the creation of new Universal Windows Platform (UWP) apps using a wizard-based experience. The resulting UWP project is well-formed, readable code that incorporates the latest Windows features while implementing proven patterns and best practices.

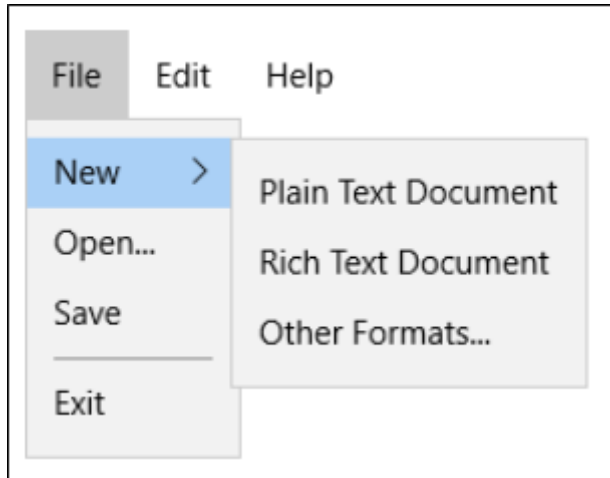


See [Windows Template Studio](#) ↗

Controls to create desktop-style UIs

We've released new UWP XAML controls that fill the gap between a traditional desktop application UI and a UWP UI.

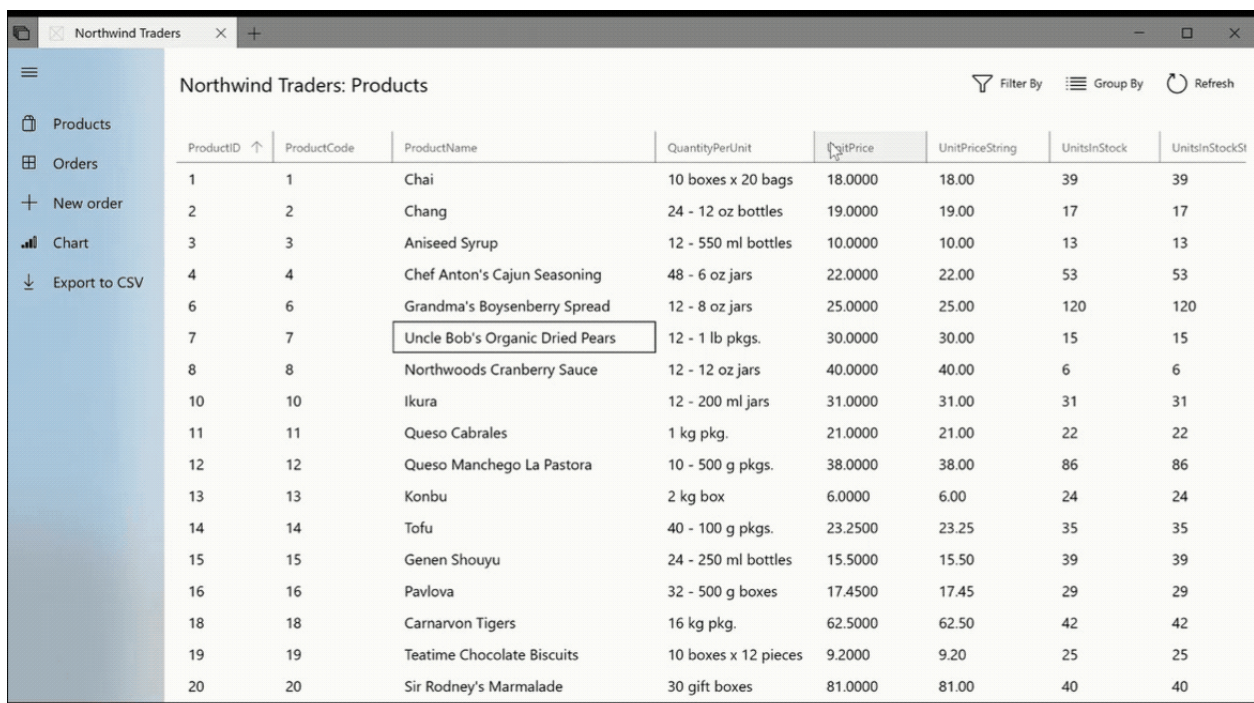
For example, the new [MenuBar](#), [DropDownButton](#), [SplitButton](#), and [CommandBarFlyout](#) controls give you more flexible ways to expose commands, and the [EditableComboBox](#) lets the user enter values that aren't listed in a pre-defined list of options.



Controls to support enterprise scenarios

The [DataGridView](#) provides a flexible way to display a collection of data in rows and columns.

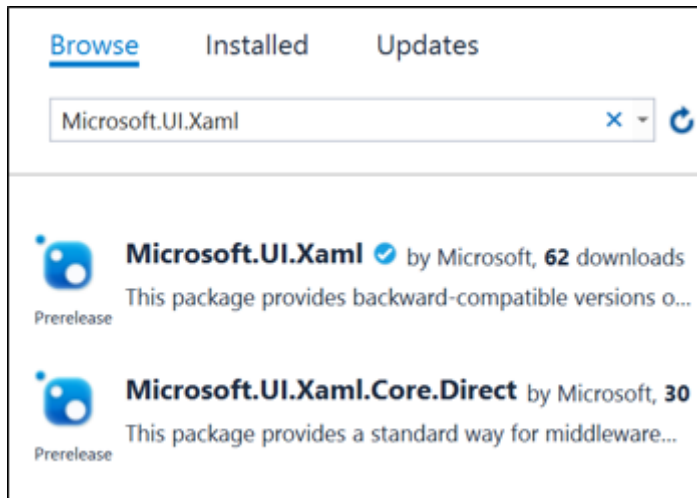
The [TreeView](#) enables a hierarchical list with expanding and collapsing nodes that contain nested items. It can be used to illustrate a folder structure or nested relationships in your UI.



ProductID	ProductCode	ProductName	QuantityPerUnit	UnitPrice	UnitPriceString	UnitsInStock	UnitsInStock
1	1	Chai	10 boxes x 20 bags	18.0000	18.00	39	39
2	2	Chang	24 - 12 oz bottles	19.0000	19.00	17	17
3	3	Aniseed Syrup	12 - 550 ml bottles	10.0000	10.00	13	13
4	4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	22.0000	22.00	53	53
6	6	Grandma's Boysenberry Spread	12 - 8 oz jars	25.0000	25.00	120	120
7	7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	30.0000	30.00	15	15
8	8	Northwoods Cranberry Sauce	12 - 12 oz jars	40.0000	40.00	6	6
10	10	Ikura	12 - 200 ml jars	31.0000	31.00	31	31
11	11	Queso Cabrales	1 kg pkg.	21.0000	21.00	22	22
12	12	Queso Manchego La Pastora	10 - 500 g pkgs.	38.0000	38.00	86	86
13	13	Konbu	2 kg box	6.0000	6.00	24	24
14	14	Tofu	40 - 100 g pkgs.	23.2500	23.25	35	35
15	15	Genen Shouyu	24 - 250 ml bottles	15.5000	15.50	39	39
16	16	Pavlova	32 - 500 g boxes	17.4500	17.45	29	29
18	18	Carnarvon Tigers	16 kg pkg.	62.5000	62.50	42	42
19	19	Teatime Chocolate Biscuits	10 boxes x 12 pieces	9.2000	9.20	25	25
20	20	Sir Rodney's Marmalade	30 gift boxes	81.0000	81.00	40	40

Windows UI Library

The Windows UI Library is a set of NuGet packages that provide controls and other user interface elements for UWP apps. It also enables down-level compatibility with earlier versions of Windows 10, so your app works even if users don't have the latest OS.



See [Windows UI Library \(Preview version\)](#).

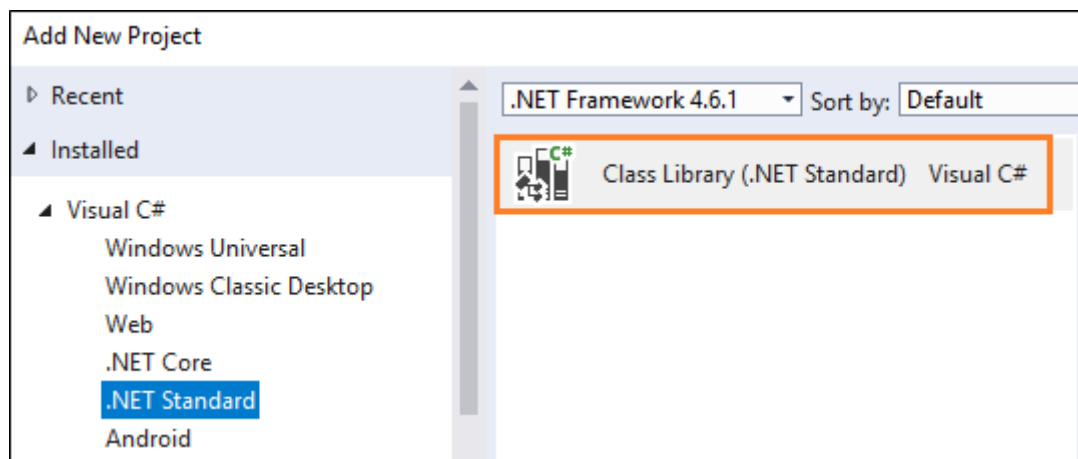
UWP controls in desktop applications (XAML Islands)

Windows 10 now enables you to use UWP controls in WPF, Windows Forms, and C++ Win32 desktop applications using a feature called *XAML Islands*. This means that you can enhance the look, feel, and functionality of your existing desktop applications with the latest Windows UI features that are only available via UWP controls, such as Windows Ink and controls that support the Fluent Design System. This feature is called XAML islands.

See [UWP controls in desktop applications](#).

.NET Standard 2.0

The .NET Standard includes over 20,000 more APIs than .NET Standard 1.x. This makes it so much easier to migrate existing .NET Framework libraries and then use them across different .NET applications including your UWP application.



See [Share code between a desktop app and a UWP app](#).

SQL Server connectivity

Your app can connect directly to a SQL Server database and then store and retrieve data by using classes in the [System.Data.SqlClient](#) namespace.

See [Use a SQL Server database in a UWP app](#).

MSIX deployment

MSIX is a Windows app package format that combines the best features of MSI, .appx, App-V, and ClickOnce to provide a modern and reliable packaging experience to all Windows apps. The MSIX package format preserves the functionality of existing app packages and install files in addition to enabling modern packaging and deployment features to Win32, WPF, and Windows Forms apps.



See [MSIX documentation](#).

Security

Windows 10 provides a suite of security features for app developers to protect the identity of their users, the security of corporate networks, and any business data stored on devices. New for Windows 10 is Microsoft Passport, an easy-to-deploy two-factor password alternative that is accessible by using a PIN or Windows Hello, which provides enterprise grade security and supports fingerprint, facial, and iris based recognition.

 Expand table

Topic	Description
Intro to secure Windows app development	This introductory article explains various Windows security features across the stages of authentication, data-in-flight, and data-at-rest. It also describes how you can integrate those stages into your apps. It covers a large range of topics, and is aimed primarily at helping app architects better understand the Windows features that make creating Universal Windows Platform apps quick and easy.
Authentication and user identity	UWP apps have several options for user authentication which are outlined in this article. For the enterprise, the new Microsoft Passport feature is strongly recommended. Microsoft Passport replaces passwords with strong two-factor authentication (2FA) by verifying existing credentials and by creating a device-specific credential that a biometric or PIN-based user gesture protects, resulting in a both convenient and highly secure experience.
Cryptography	The cryptography section provides an overview of the cryptography features available to UWP apps. Articles range from introductory walkthroughs on how to easily encrypt sensitive business data, to advanced to advanced topics such as manipulating cryptographic keys and working with MACs, hashes, and signatures.
Windows Information Protection (WIP)	This is a hub topic covering the full developer picture of how Windows Information Protection (WIP) relates to files, buffers, clipboard, networking, background tasks, and data protection under lock.

Data binding and databases

Data binding is a way for your app's UI to display data from an external source, such as a database, and optionally to stay in sync with that data. Data binding allows you to separate the concern of data from the concern of UI, and that results in a simpler conceptual model as well as better readability, testability, and maintainability of your app.

 Expand table

Topic	Description
Data binding overview	This topic shows you how to bind a control (or other UI element) to a single item or bind an items control to a collection of items in a Universal Windows Platform (UWP) app. In addition, it shows how to control the rendering of items, implement a details view based on a selection, and convert data for display.
Entity Framework 7 for UWP	Performing complex queries against large data sets is vastly simplified using Entity Framework 7, which supports UWP. In this walkthrough, you will build a UWP app that performs basic data access against a local SQLite database using Entity Framework.
SQLite local database	SQLite is the recommended solution for local app databases. Visit SQLite to download the latest version for UWP, or use the version that's already provided with the Windows SDK.

Networking and data serialization

Line-of-business apps often need to communicate with or store data on a variety of other systems. This is typically accomplished by connecting to a network service (using protocols such as REST or SOAP) and then serializing or deserializing data into a common format. Working with networks and data serialization in UWP apps similar to WPF, WinForms, and ASP.NET applications. See the following articles for more information.

 Expand table

Topic	Description
Networking basics	This walkthrough explains basic networking concepts relevant to all UWP apps, regardless of the communication protocols in use.
Which networking technology?	A quick overview of the networking technologies available for UWP apps, with suggestions on how to choose the technologies that are the best fit for your app.
XML and SOAP serialization	XML serialization converts objects into an XML stream that conforms to a specific XML Schema definition language (XSD). To convert between XML and a strongly-typed class, you can use the native XDocument class, or an external library.
JSON serialization	JSON (JavaScript object notation) serialization is a popular format for communicating with REST APIs. The Newtonsoft Json.NET , which is fully supported for UWP apps.

Devices

In order to integrate with line-of-business tools, like printers, barcode scanners, or smart card readers, you may find it necessary to integrate external devices or sensors into your app. Here are some examples of features that you can add to your app using the technology described in this section.

 Expand table

Topic	Description
Enumerate devices	This article explains how to use the Windows.Devices.Enumeration namespace to find devices that are internally connected to the system, externally connected, or detectable over wireless or networking protocols. Start here if you're building any app that works with devices.
Printing and scanning	Describes how to print and scan from your app, including connecting to and working with business devices like point-of-sale (POS) systems, receipt printers, and high-capacity feeder scanners.
Bluetooth	In addition to using traditional Bluetooth connections to send and receive data or control devices, Windows 10 enables using Bluetooth Low Energy (BTLE) to send or receive beacons in the background. Use this to display notifications or enable functionality when a user gets close to or leaves a particular location.
Enterprise shared storage	In device lockdown scenarios, learn how data can be shared within the same app, between instances of an app, or even between apps.

Device targeting

Many users today are bringing their own phone or tablet to work, which have varying form factors and screen sizes. With the Universal Windows Platform (UWP), you can write a single line-of-business app that runs seamlessly on all different types of devices, including desktop PCs and PPI displays, allowing you to maximize the reach of your app and the efficiency of your code.

 Expand table

Topic	Description
Guide to UWP apps	In this introductory guide, you'll get acquainted with the Windows UWP platform, including: what a device family is and how to decide which one to target, new UI controls and panels that allow you to adapt your UI to different device form

Topic	Description
	factors, and how to understand and control the API surface that is available to your app.
Adaptive XAML UI code sample 	This code sample shows all the possible layout options and controls for your app, regardless of device type, and allows you to interact with the panels to show how to achieve any layout you are looking for. In addition to showing how each control responds to different form factors, the app itself is responsive and shows various methods for achieving adaptive UI.
Xamarin topic	Xamarin for targeting phone

Deployment

You have options for distributing apps to your organization's users using MSIX packages. You can configure an App Installer-based deployment, use device management tools such as Microsoft Endpoint Configuration Manager and Microsoft Intune, publish to the Microsoft Store for Business, or you can sideload apps to devices. You can also make your apps available to the general public by publishing to the Microsoft Store.

 Expand table

Topic	Description
MSIX documentation	MSIX is a Windows app package format that combines the best features of MSI, .appx, App-V, and ClickOnce to provide a modern and reliable packaging experience.
Distribute LOB apps to enterprises	Learn about the options for distributing line-of-business apps without making the apps broadly available to the public, including App Installer-based deployment, Microsoft Endpoint Configuration Manager and Microsoft Intune, and publishing to the Microsoft Store for Business.
Sideload apps	When you sideload an app, you deploy a signed app package to a device. You maintain the signing, hosting, and deployment of these apps. The process for sideloading apps is streamlined for Windows.
Publish apps to the Microsoft Store	The unified Microsoft Store lets you publish and manage all of your apps for all Windows devices. Customize your app's availability with per-market pricing, distribution and visibility controls, and other options.

Enterprise UWP samples

Topic	Description
VanArsdel Inventory Sample 	A UWP sample app that showcases line-of-business scenarios. The sample is based around creating and managing customer, orders, and products for the fictitious company VanArsdel.
Customer Orders Database Sample 	A UWP sample app that showcases features useful to enterprise developers, like Azure Active Directory (AAD) authentication, UI controls (including a data grid), Sqlite and SQL Azure database integration, Entity Framework, and cloud API services. The sample is based around creating and managing customer accounts, orders, and products for the fictitious company Contoso.

Patterns and practices

Code bases for large scale, enterprise-grade apps can become unwieldy. Prism is a framework for building loosely coupled, maintainable, and testable XAML applications in WPF, Windows 10 UWP, and Xamarin Forms. Prism provides an implementation of a collection of design patterns that are helpful in writing well-structured and maintainable XAML applications, including MVVM, dependency injection, commands, EventAggregator, and others.

For more information on Prism, see the [GitHub repo](#) .

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Windows Information Protection (WIP)

Article • 10/27/2022

Note Windows Information Protection (WIP) policy can be applied to Windows 10, version 1607 or later.

WIP protects data that belongs to an organization by enforcing policies that are defined by the organization. If your app is included in those policies, all data produced by your app is subject to policy restrictions. This topic helps you to build apps that more gracefully enforce these policies without having any impact on the user's personal data.

First, what is WIP?

WIP is a set of features on desktops, laptops, tablets, and phones that support the organization's Mobile Device Management (MDM) and Mobile Application Management (MAM) system.

WIP together with MDM gives the organization greater control over how its data is handled on devices that the organization manages. Sometimes users bring devices to work and do not enroll them in the organization's MDM. In those cases, organizations can use MAM to achieve greater control over how their data is handled on specific line of business apps that users install on their device.

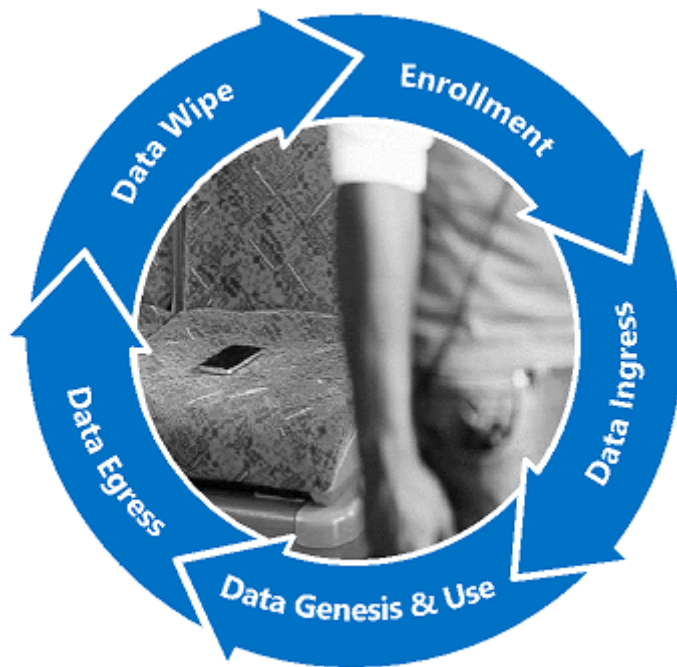
using MDM or MAM, administrators can identify which apps are allowed to access files that belong to the organization and whether users can copy data from those files and then paste that data into personal documents.

Here's how it works. Users enroll their devices into the organization's mobile device management (MDM) system. An administrator in the managing organization uses Microsoft Intune or System Center Configuration Manager (SCCM) to define and then deploy a policy to the enrolled devices.

If users aren't required to enroll their devices, administrators will use their MAM system to define and deploy a policy that applies to specific apps. When users install any of those apps, they'll receive the associated policy.

That policy identifies the apps that can access enterprise data (called the policy's *allowed list*). These apps can access enterprise protected files, Virtual Private Networks (VPN) and enterprise data on the clipboard or through a share contract. The policy also defines the rules that govern the data. For example, whether data can be copied from enterprise-owned files and then pasted into non enterprise-owned files.

If users unenroll their device from the organization's MDM system, or uninstall apps identified by the organizations MAM system, administrators can remotely wipe enterprise data from the device.



Read more about WIP

- [Introducing Windows Information Protection](#) 
- [Protect your enterprise data using Windows Information Protection \(WIP\)](#)

If your app is on the allowed list, all data produced by your app is subject to policy restrictions. That means that if administrators revoke the user's access to enterprise data, those users lose access to all of the data that your app produced.

This is fine if your app is designed only for enterprise use. But if your app creates data that users consider personal to them, you'll want to *enlighten* your app to intelligently discern between enterprise and personal data. We call this type of an app *enterprise-enlightened* because it can gracefully enforce enterprise policy while preserving the integrity of the user's personal data.

Create an enterprise-enlightened app

Use WIP APIs to enlighten your app, and then declare your app as enterprise-enlightened.

Enlighten your app if it'll be used for both organizational and personal purposes.

Enlighten your app if you want to gracefully handle the enforcement of policy elements.

For example, if policy allows users to paste enterprise data into a personal document, you can prevent users from having to respond to a consent dialog before they paste the data. Similarly, you can present custom informational dialog boxes in response to these sorts of events.

If you're ready to enlighten your app, see one of these guides:

For Universal Windows Platform (UWP) apps that you build by using C#

[Windows Information Protection \(WIP\) developer guide.](#)

For Desktop apps that you build by using C++

[Windows Information Protection \(WIP\) developer guide \(C++\).](#)

Create non-enlightened enterprise app

if you're creating a Line of Business (LOB) app that is not intended for personal use, you might not have to enlighten it.

Windows desktop apps

You don't need to enlighten a Windows desktop app but you should test your app to ensure that it functions properly under policy. For example, start your app, use it, then unenroll the device from MDM. Then, make sure the app can start again. If files critical to the operation of the app are encrypted, the app might not start. Also, review the files that your app interacts with to ensure that your app won't inadvertently encrypt files that are personal to the user. This might include metadata files, images and other things.

After you've tested your app, add this flag to the resource file or your project, and then recompile the app.

C++

```
MICROSOFTEDPAUTOPROTECTIONALLOWEDAPPINFO EDPAUTOPROTECTIONALLOWEDAPPINFOID  
BEGIN  
    0x0001  
END
```

While MDM policies don't require the flag, MAM policies do.

UWP apps

If you expect your app to be included in a MAM policy, you should enlighten it. Policies deployed to devices under MDM won't require it, but if you distribute your app to organizational consumers, it's difficult if not impossible to determine what type of policy management system they'll use. To ensure that your app will work in both types of policy management systems (MDM and MAM), you should enlighten your app.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)